

EECS 836: Machine Learning

Zijun Yao

Assistant Professor, EECS Department
The University of Kansas

Agenda

- Convolutional Neural Networks (CNN)
 - What is CNN
 - Convolutional Layer
 - Pooling Layer
 - Backpropagation

Stable diffusion

- **Prompt:** A beautifully composed magical world, emotional and smiling girl, half body shot, environmental candid of a magic water and fire particles, with captivating, detailed eyes, natural imperfections. taken in a rainforest.



Computer vision



Self-driving vehicles

Computer vision



Face ID

Image recognition

airplane



automobile



bird



cat



deer



dog



frog



horse



ship



truck



[CIFAR-10 tiny image dataset with 10 classes](#)

The MNIST Dataset

label = 5



label = 0



label = 4



label = 2



label = 1



label = 3



label = 3



label = 5

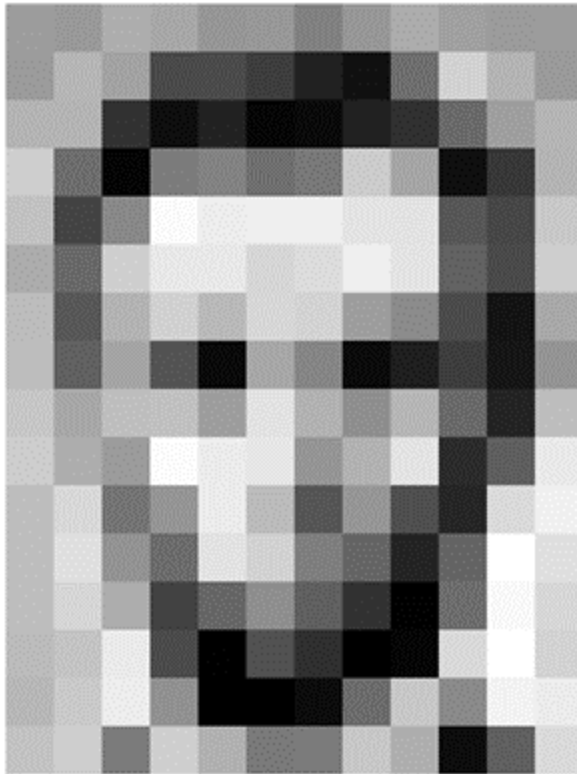


label = 3



[MNIST handwritten digit Dataset](#)

Images are matrices of numbers

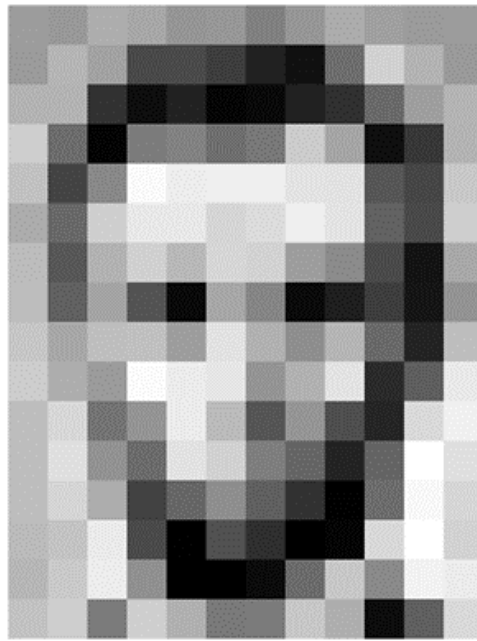


157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	106	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

An image is a matrix of numbers $[0, 255]$ (color intensity)

Classification of image



157	153	174	168	150	152	129	151	172	161	155	156
156	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	228	227	87	71	201	
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	236	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218



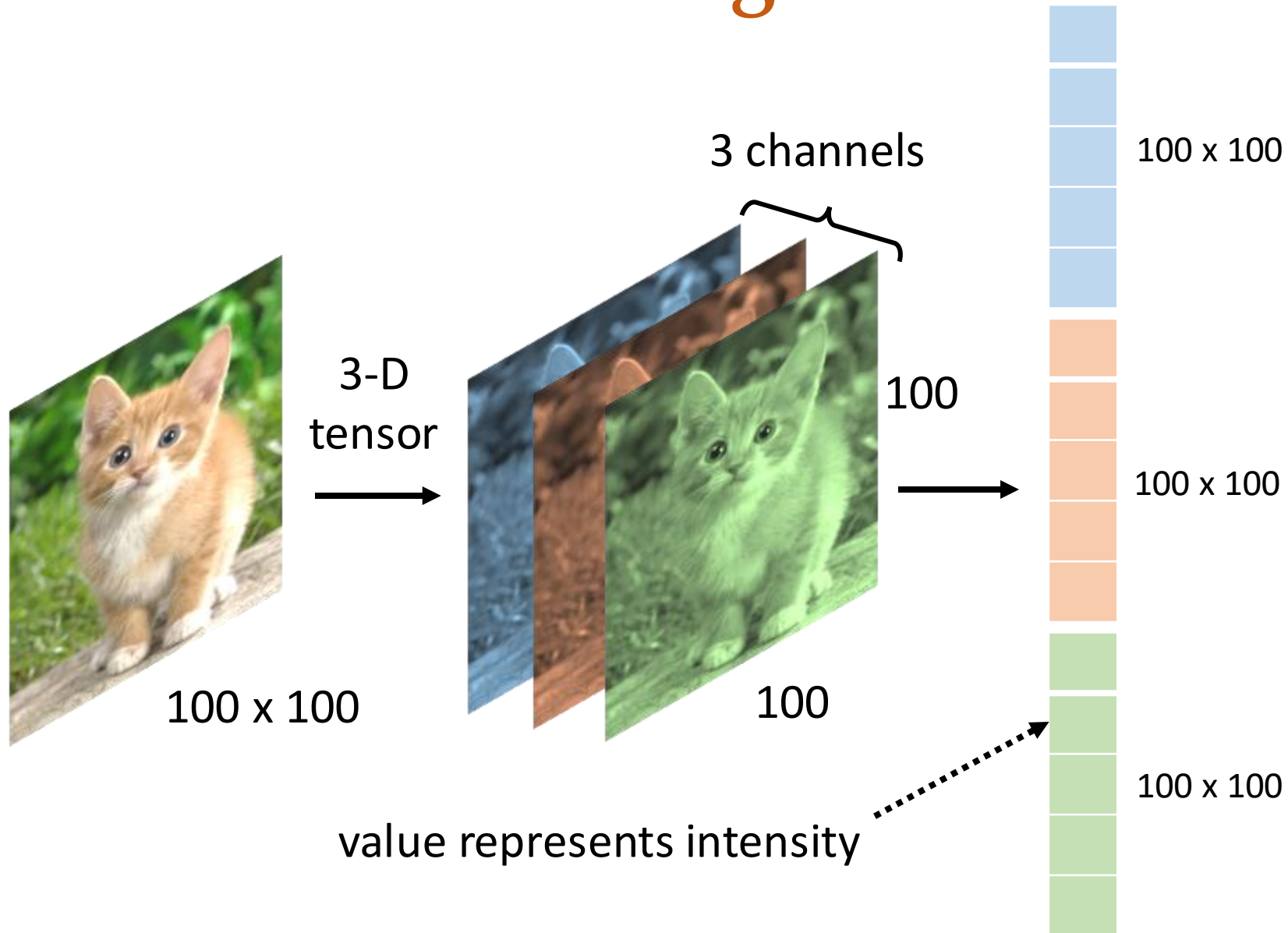
Lincoln	0.8
Washington	0.1
Jefferson	0.05
Obama	0.05

y $\longleftrightarrow$ $\hat{y}$

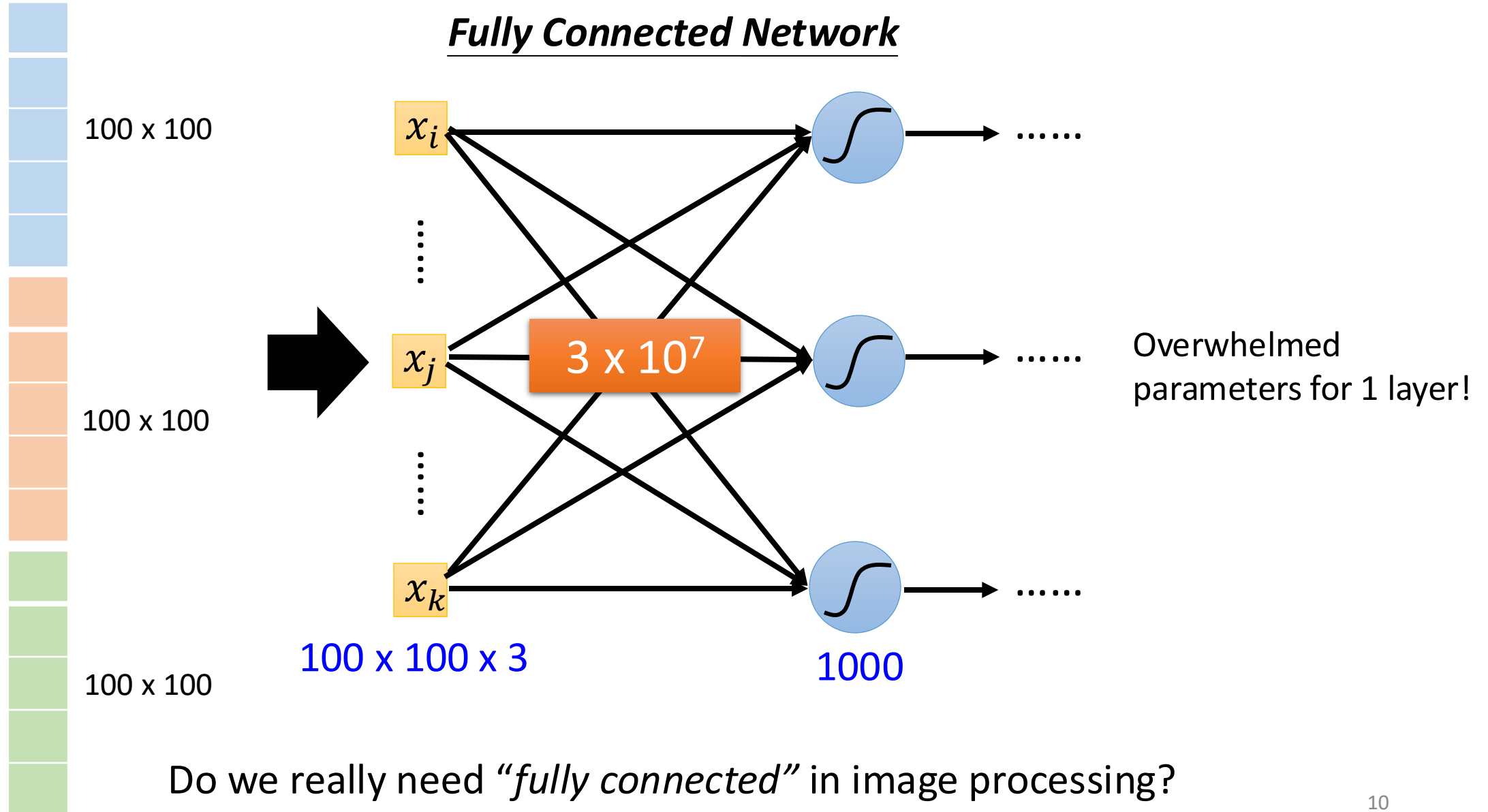
Cross entropy

Classification: predict probability of belonging to a particular class

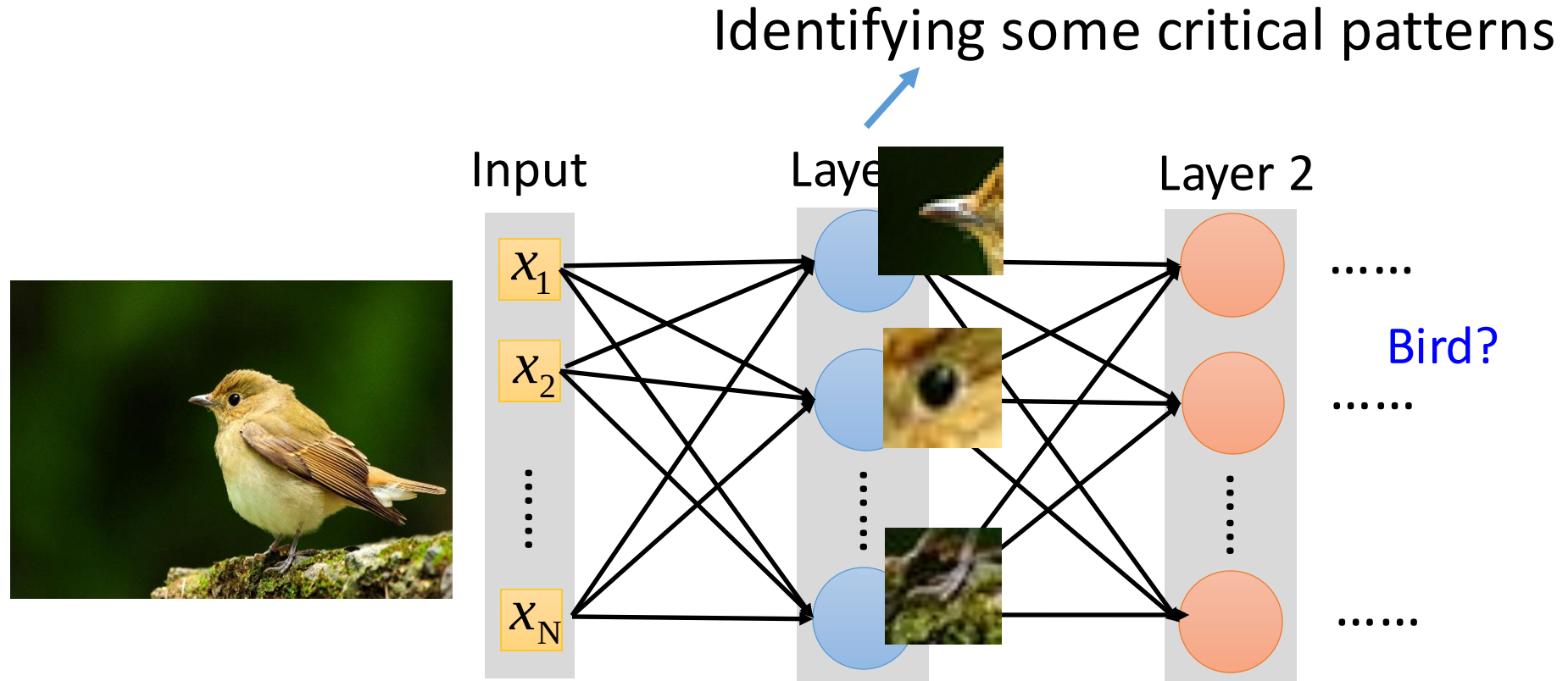
Classification of with regular NN



Classification of with regular NN



Observation



Our human don't need all the pixels to identify a bird

Why CNN for image learning (1/3)

- Some patterns are much smaller than the whole image

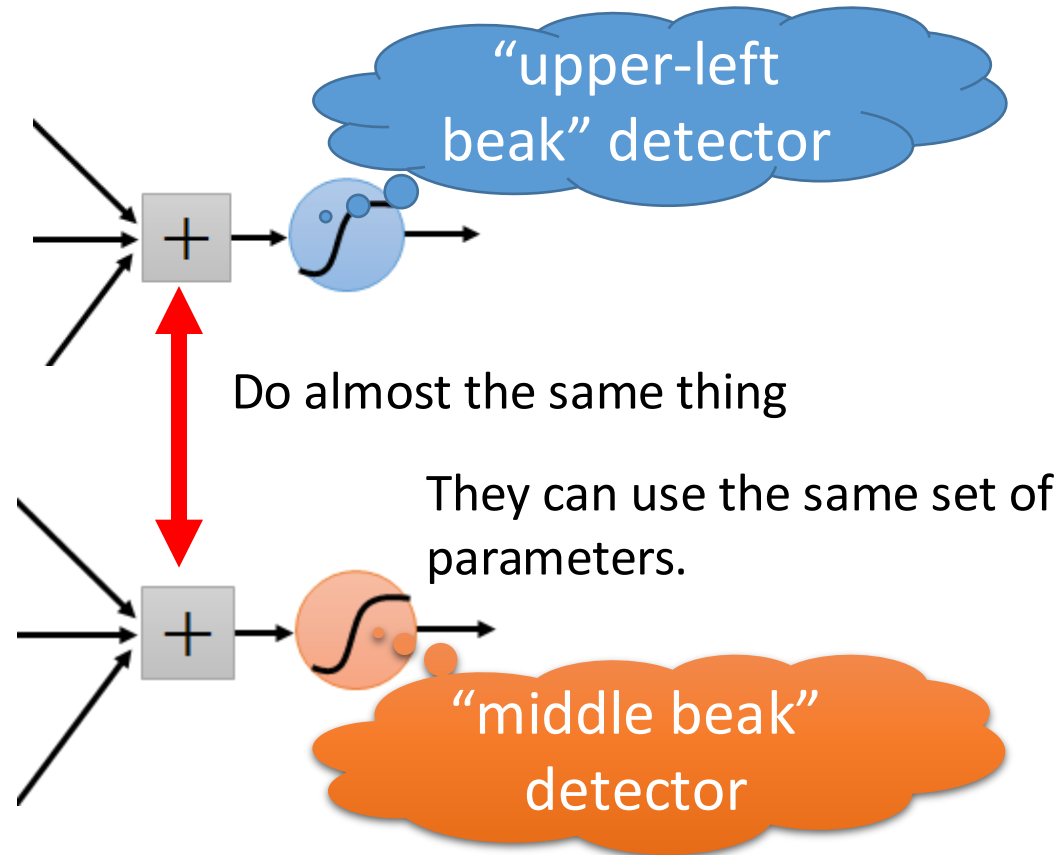
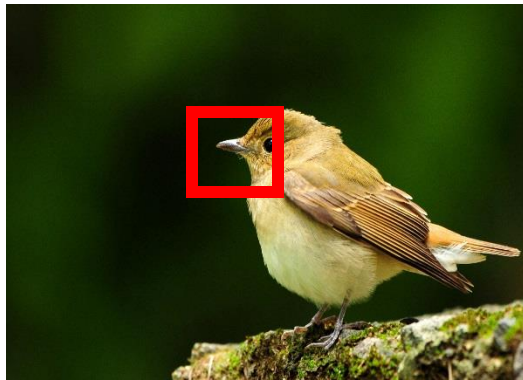
A neuron does not have to see the whole image to discover the pattern.

Connecting to small region with less parameters



Why CNN for image learning (2/3)

- The same patterns appear in different regions.



Why CNN for image learning (3/3)

- Subsampling the pixels will not change the object
 - Pictures now come with megapixel which means millions of pixels



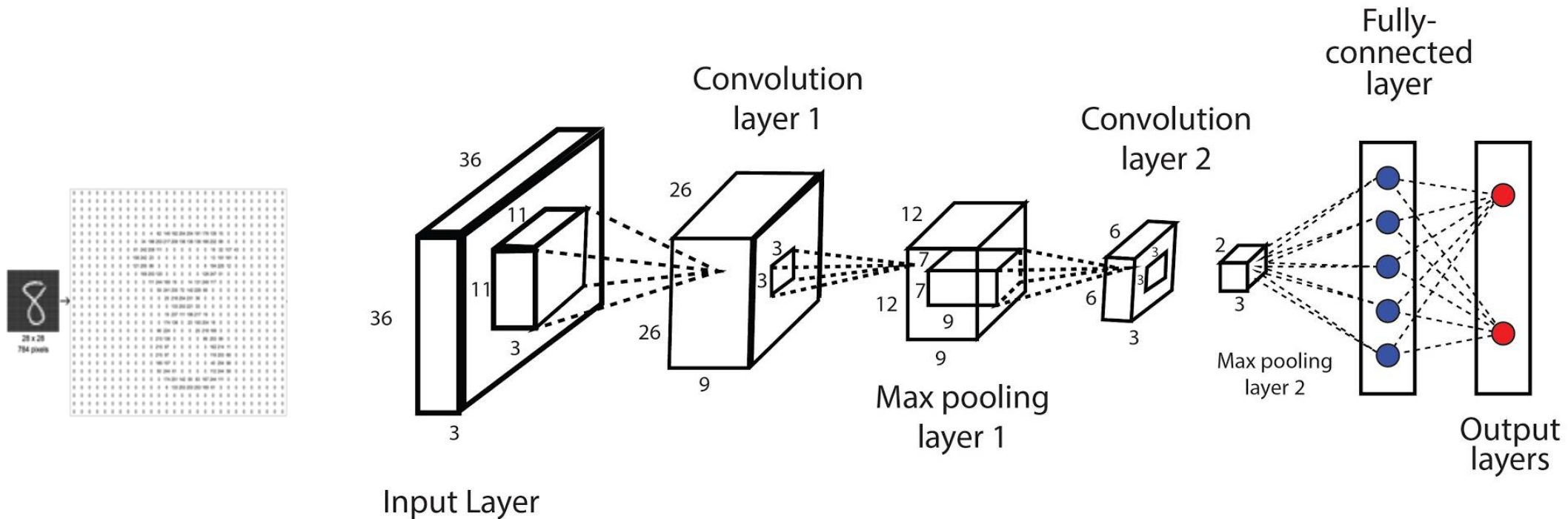
We can subsample the pixels to make image smaller

➡ Less parameters for the network to process the image

Why CNN for image learning

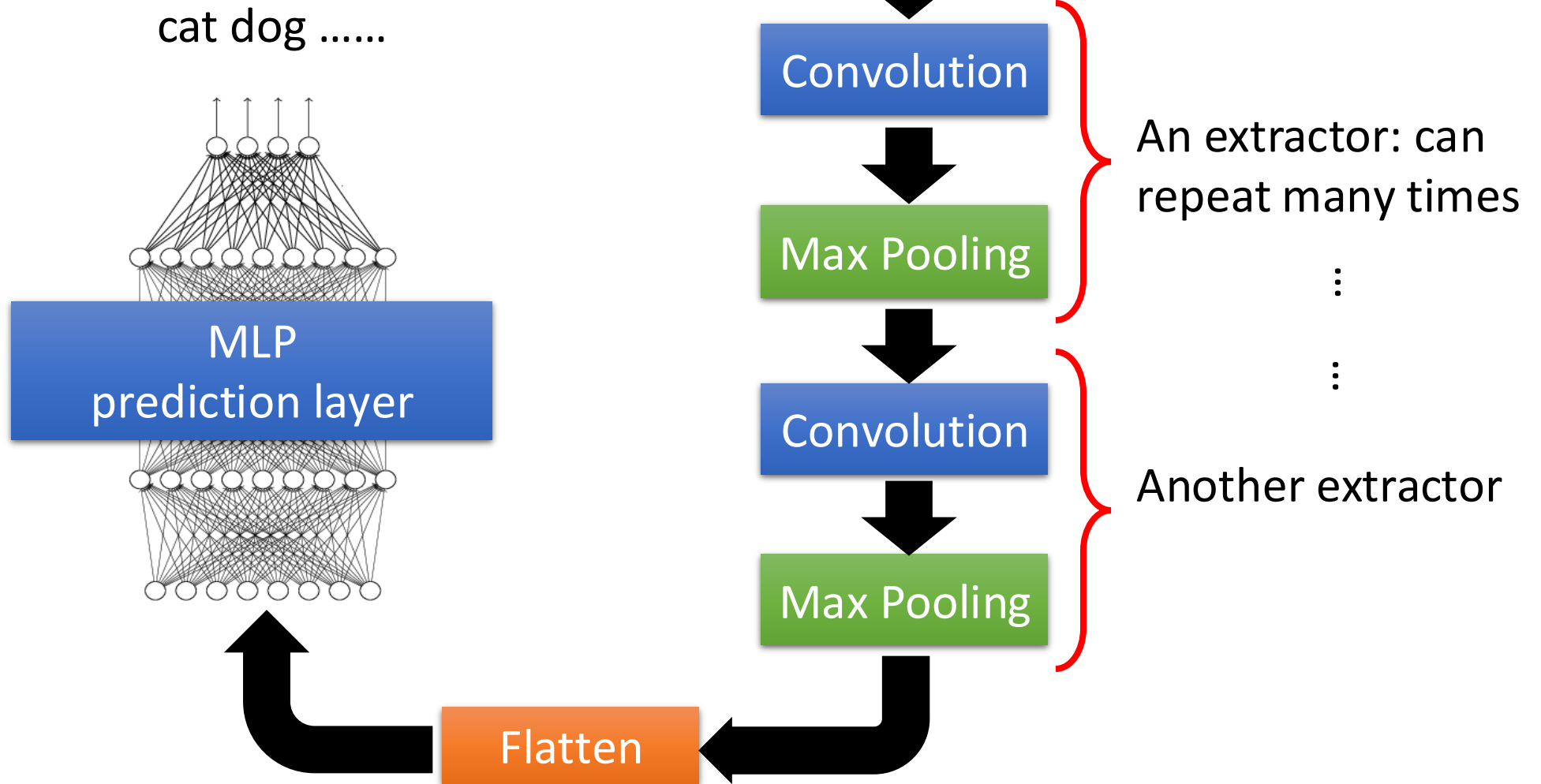
- Locality
 - Network should focus on **local regions**, without regard for the contents of the image in distant regions
 - These local representations can be **aggregated** to make predictions at the whole image level
- Translation invariance
 - Network should respond similarly to the same patch, regardless of where it appears in the image
- Subsampling
 - Less parameters for the network to process the image

A CNN structure

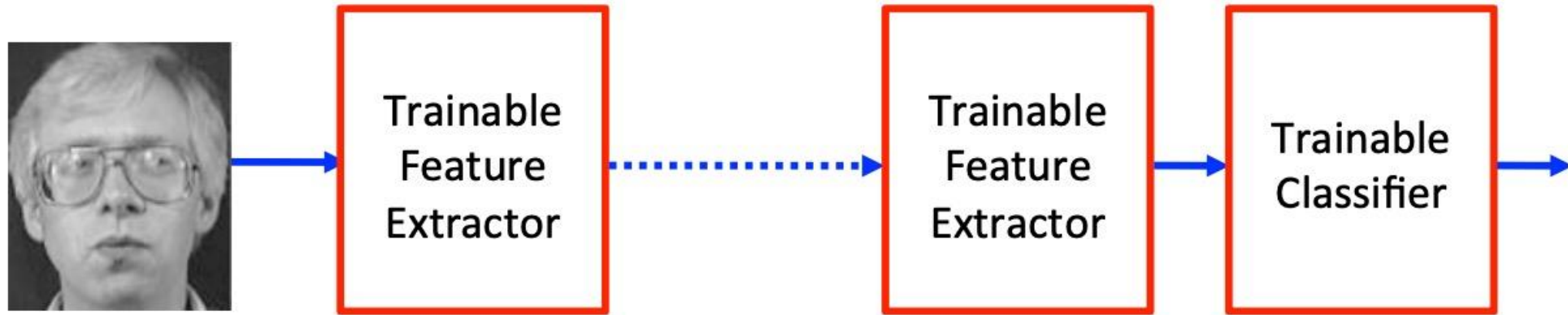


Three main types of layers: **Convolutional Layer**, **Pooling Layer**, and **Fully-Connected Layer**

The whole CNN



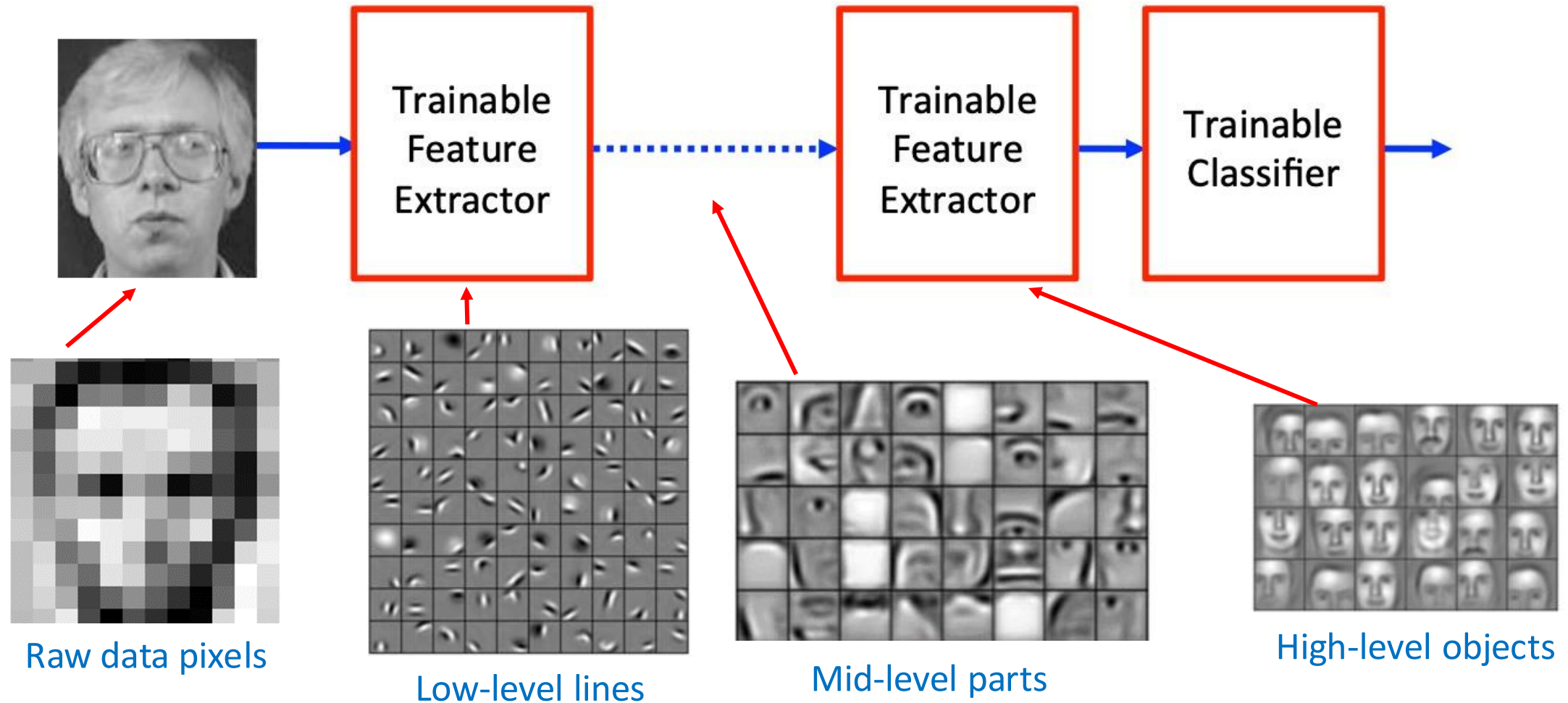
Hierarchical representation



Good image learning is hierarchical!

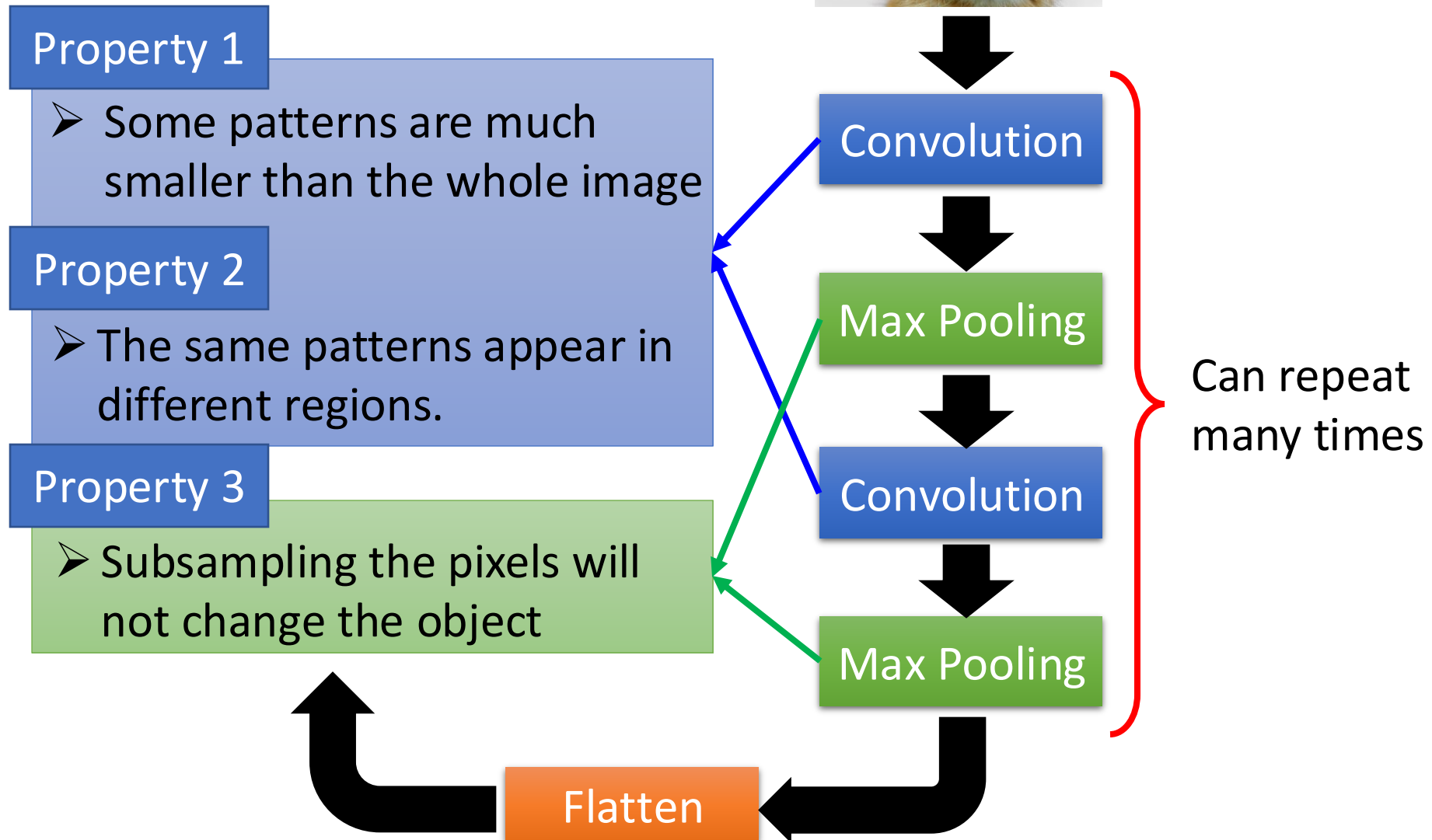
- To recognize a picture: part-to-whole hierarchy
 - Pixels ->Edges->Textons->Parts->Objects->Scenes
- Deep learning
 - Each extractor learns a hierarchy of representation at an internal layer
 - From low-level feature to mid-level representation to object identities

Hierarchical representation by convolution



Good image learning is hierarchical – each level assemble the features from previous layer!

The whole CNN

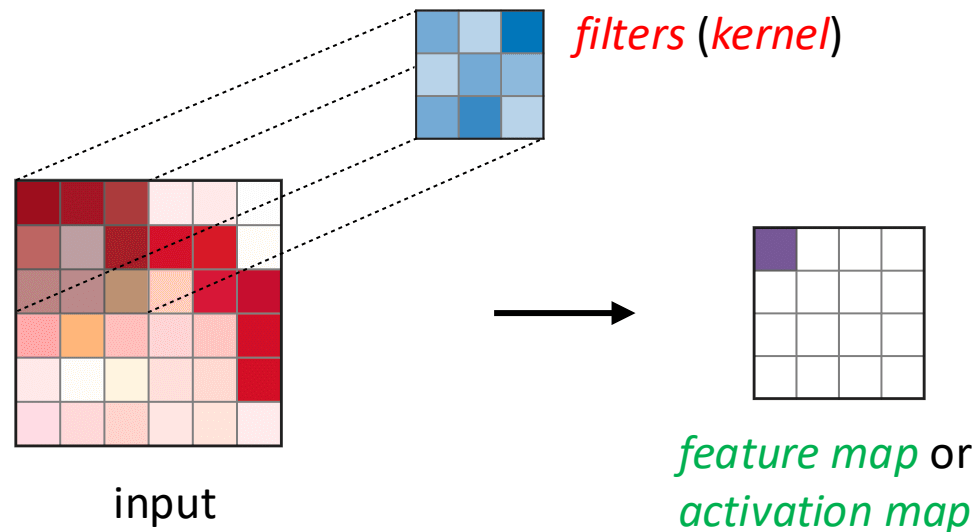


Agenda

- Convolutional Neural Networks (CNN)
 - What is CNN
 - Convolutional Layer
 - Pooling Layer
 - Backpropagation

Convolution layer

- The convolution layer uses *filters* that perform convolution operations
 - It scan the input I with respect to its dimensions.
- Its hyperparameters include the *filter size* F and *stride* S .
- The resulting output O is called *feature map* or *activation map*.



Input		Kernel		Output																	
<table border="1"><tr><td>0</td><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td><td>5</td></tr><tr><td>6</td><td>7</td><td>8</td></tr></table>	0	1	2	3	4	5	6	7	8	*	<table border="1"><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table border="1"><tr><td>19</td><td>25</td></tr><tr><td>37</td><td>43</td></tr></table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

CNN – convolution

Those are the network parameters to be learned.

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1
Matrix

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2
Matrix

⋮

Property 1

Each filter detects a small pattern (3 x 3).

CNN – convolution

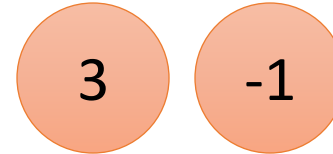
stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1



CNN – convolution

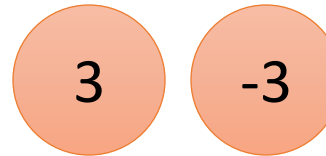
If stride=2

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1



We set stride=1 below

CNN – convolution

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

Property 2

CNN – Convolution

stride=1

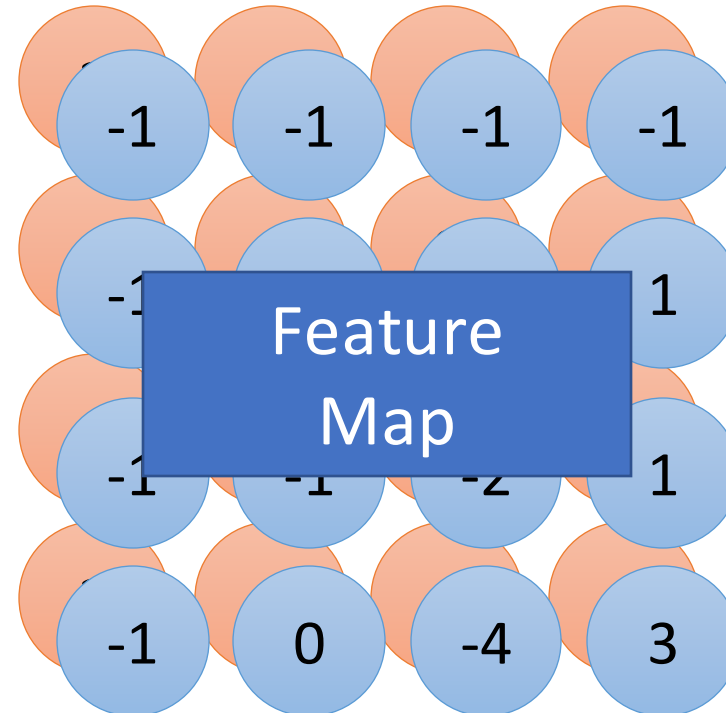
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

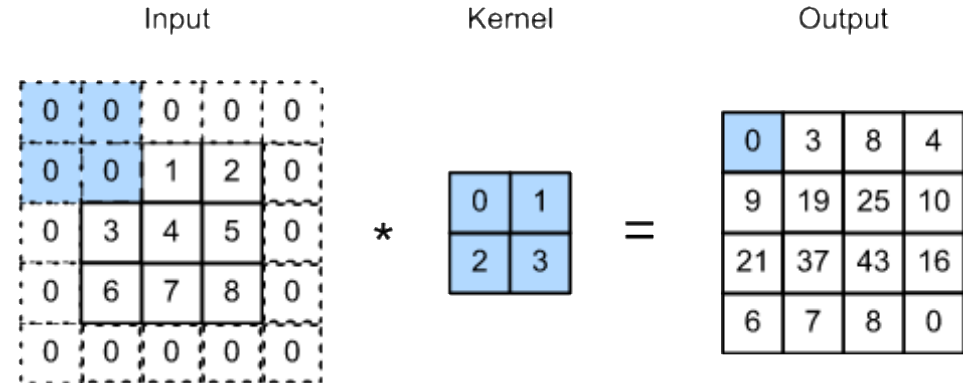
Do the same process for every filter



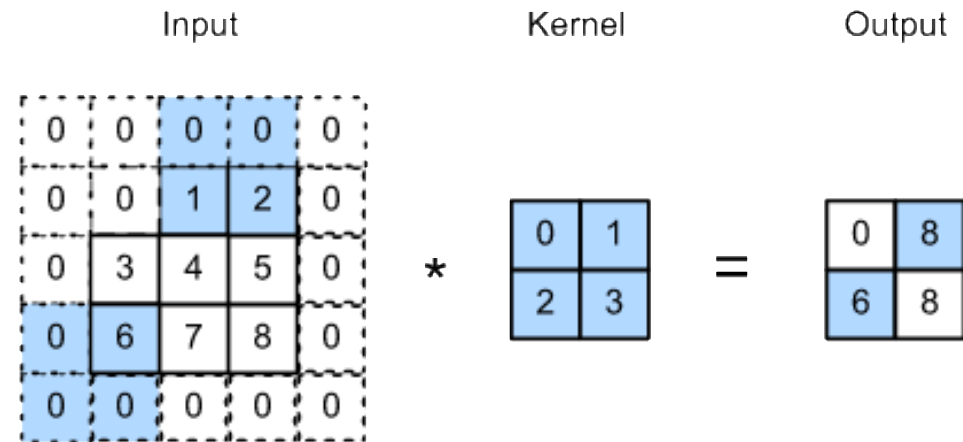
4 x 4 image

Padding and stride

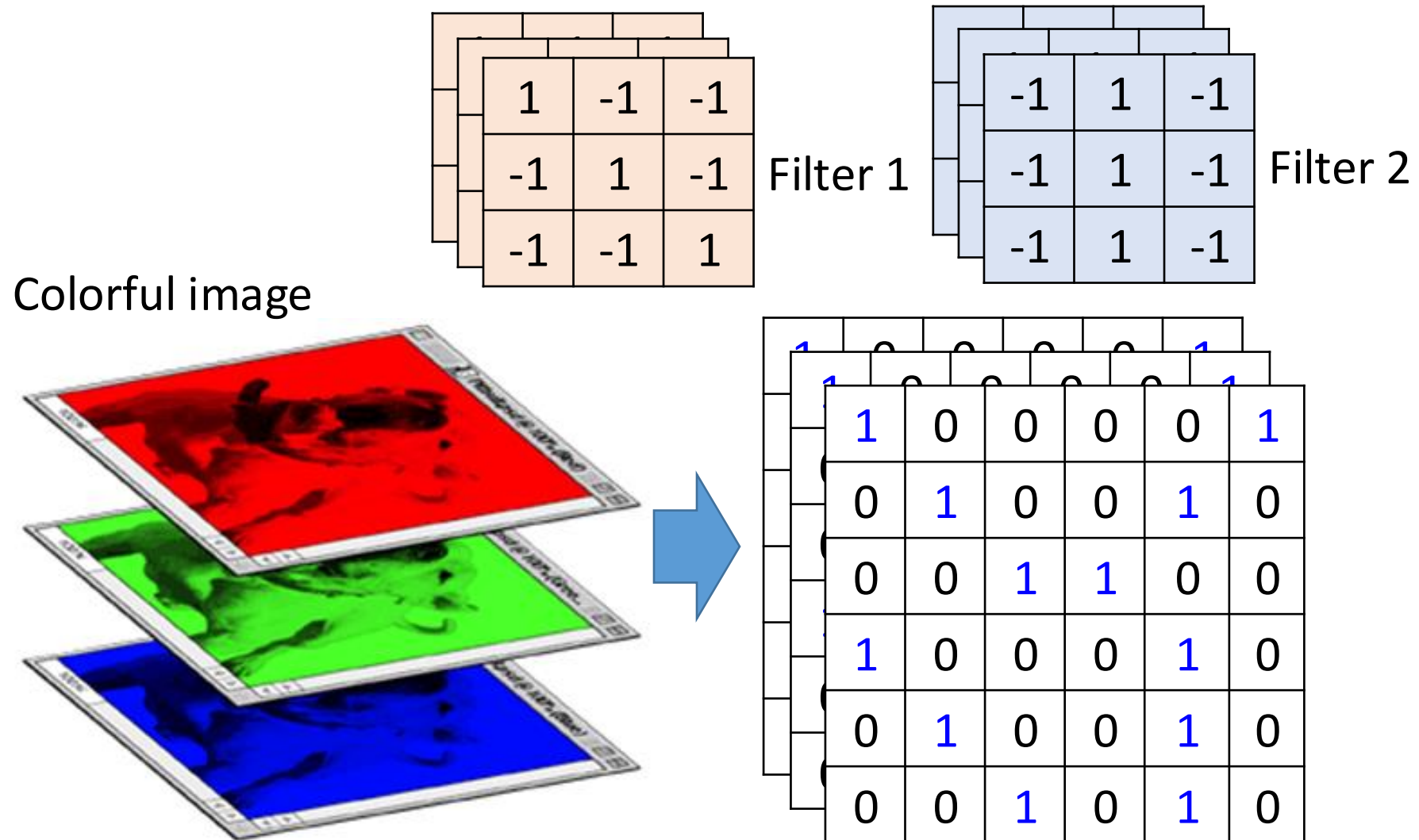
- **Padding**: add extra pixels of filler around the boundary of our input image, thus increasing the effective size of the image



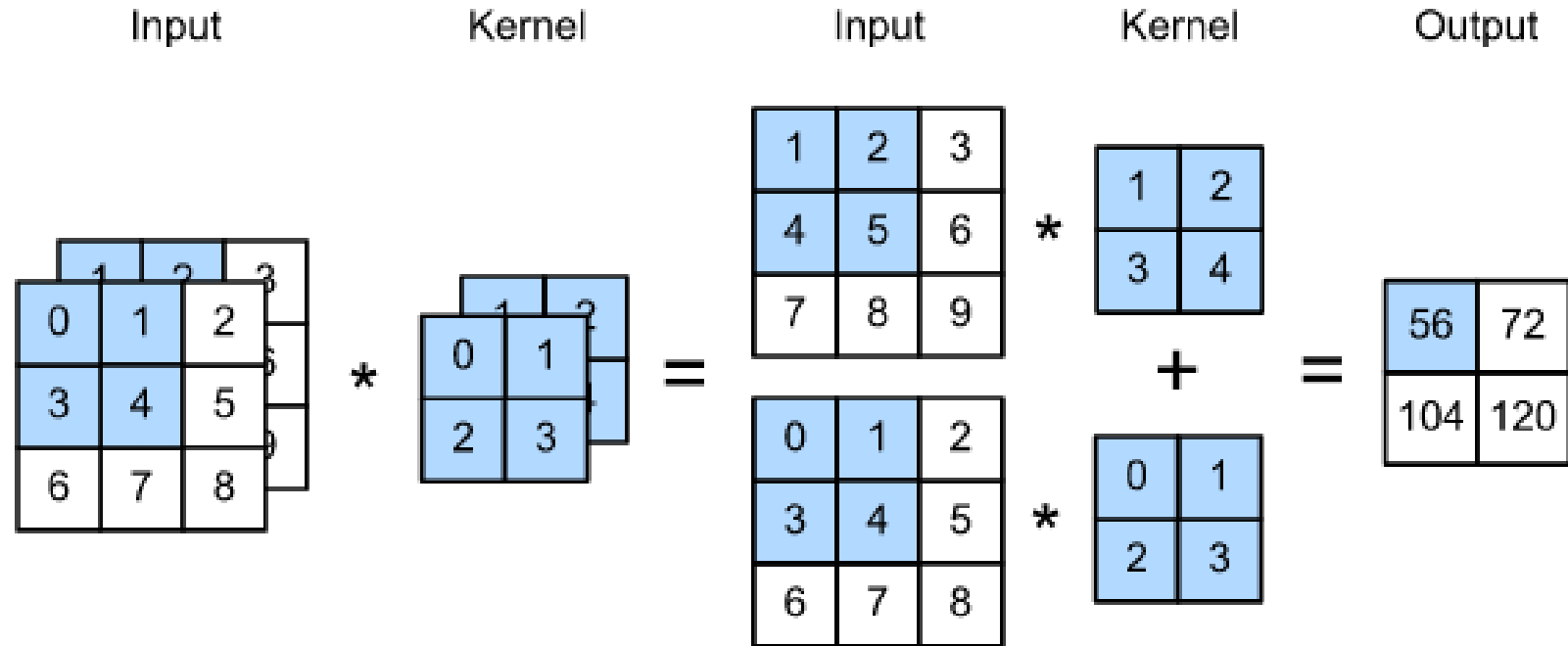
- **Stride**: the number of rows and columns traversed per slide as the *stride*



CNN – colorful image

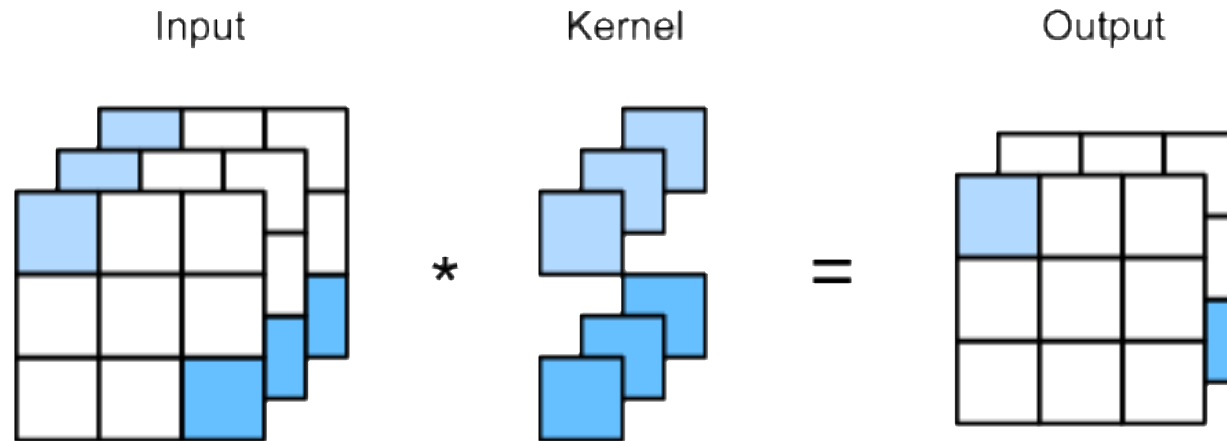


Multiple input channels



When the input data contain multiple channels, we need to construct a convolution kernel with the same number of **input channels** as the input data

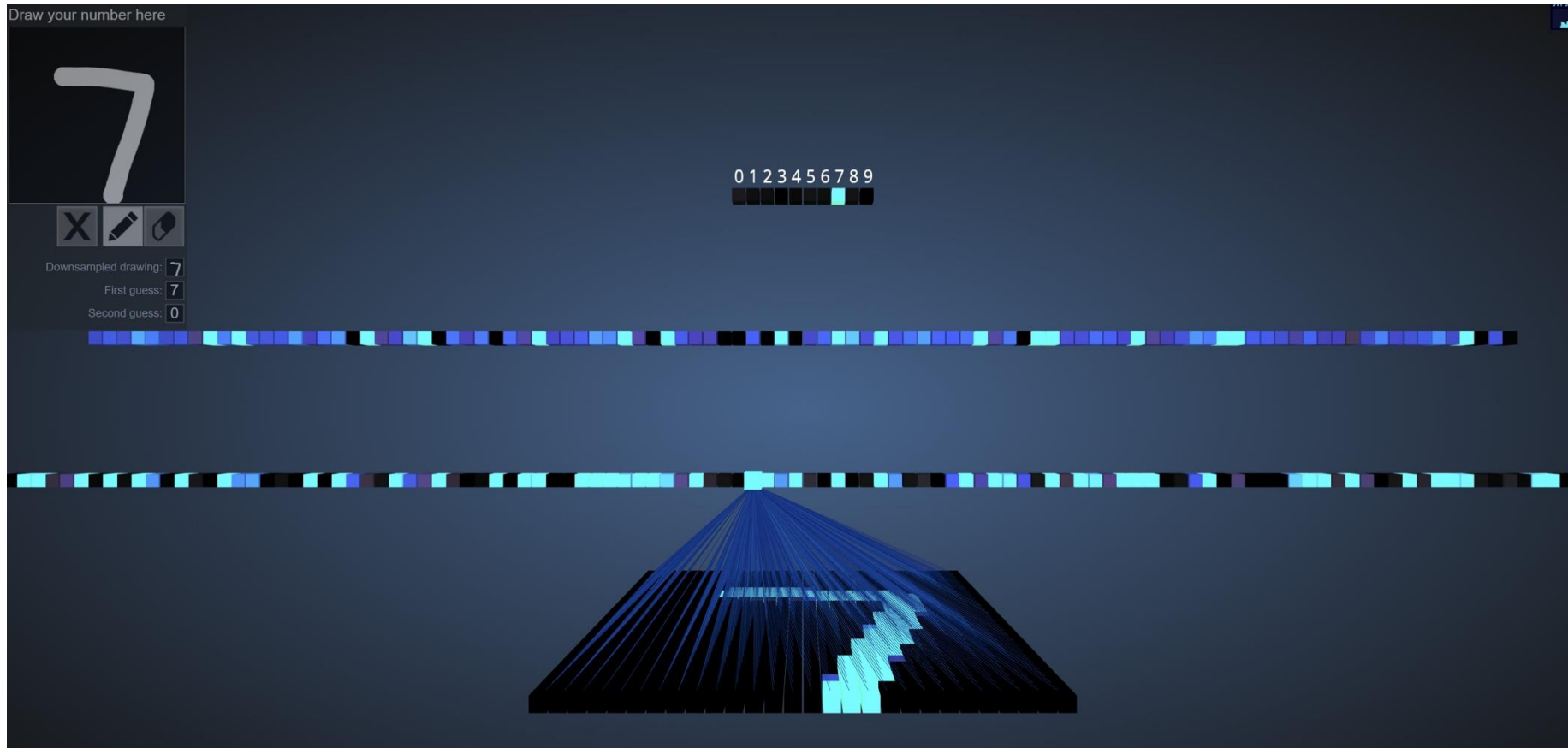
Multiple output channels



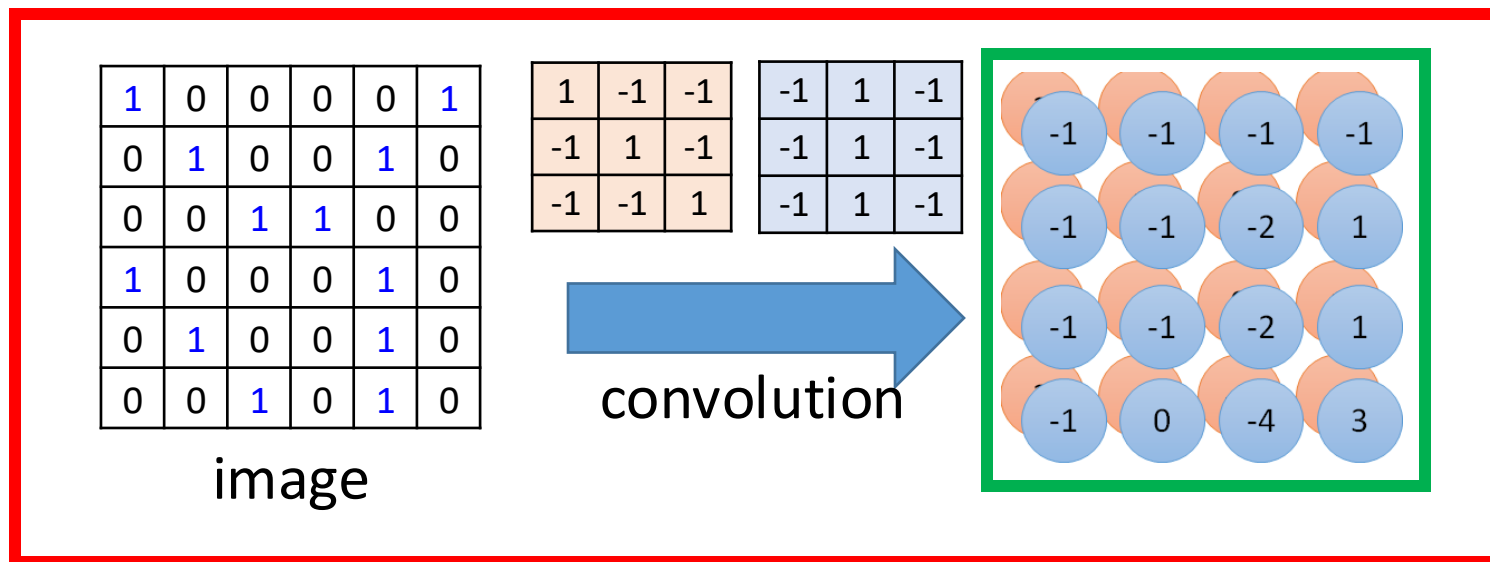
When the output contain multiple channels, we need to construct multiple convolution kernels with the same number of channels of the outputs

Convolution v.s. Fully Connected

- Visualizer: https://adamharley.com/nn_vis/

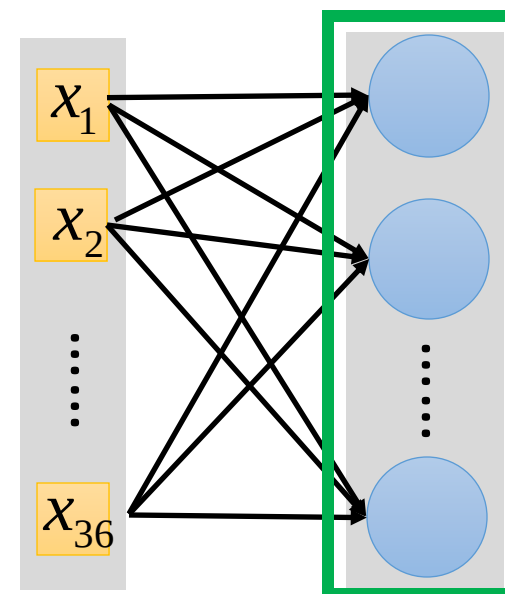


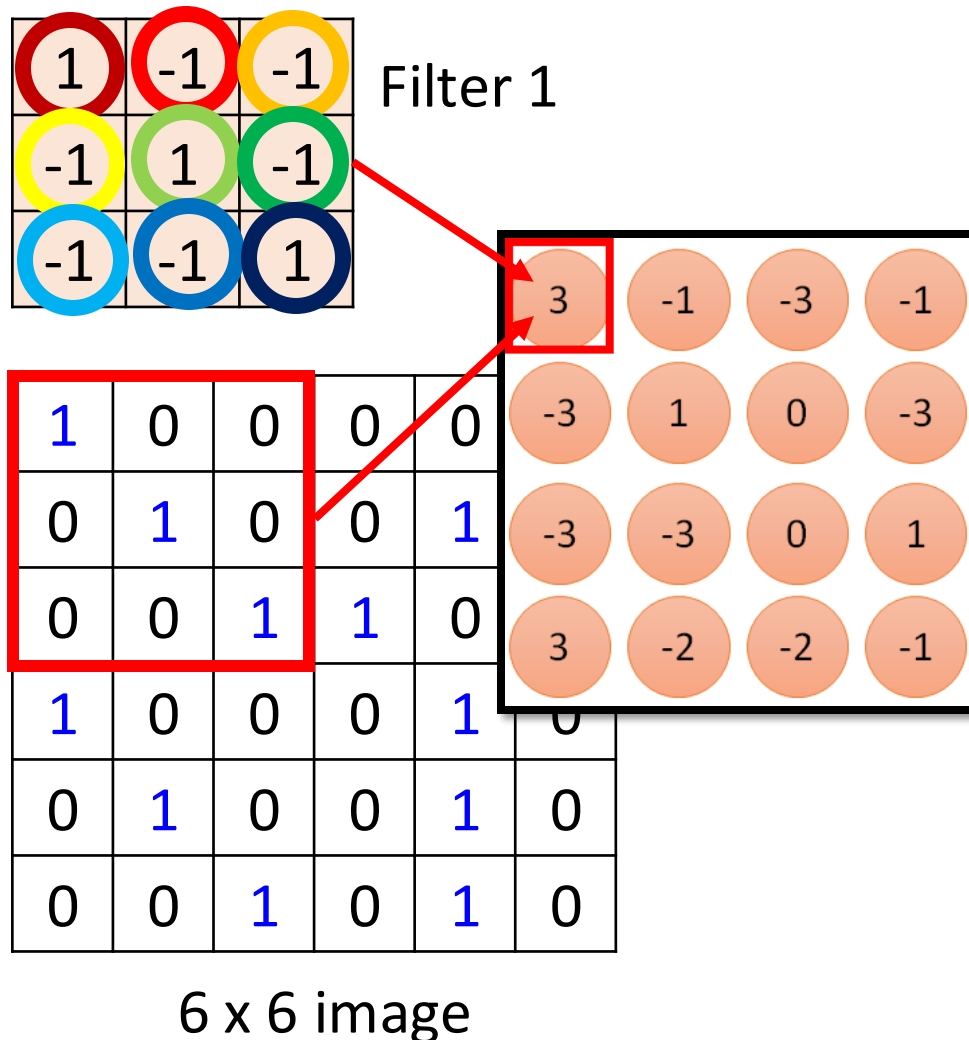
Convolution v.s. Fully Connected



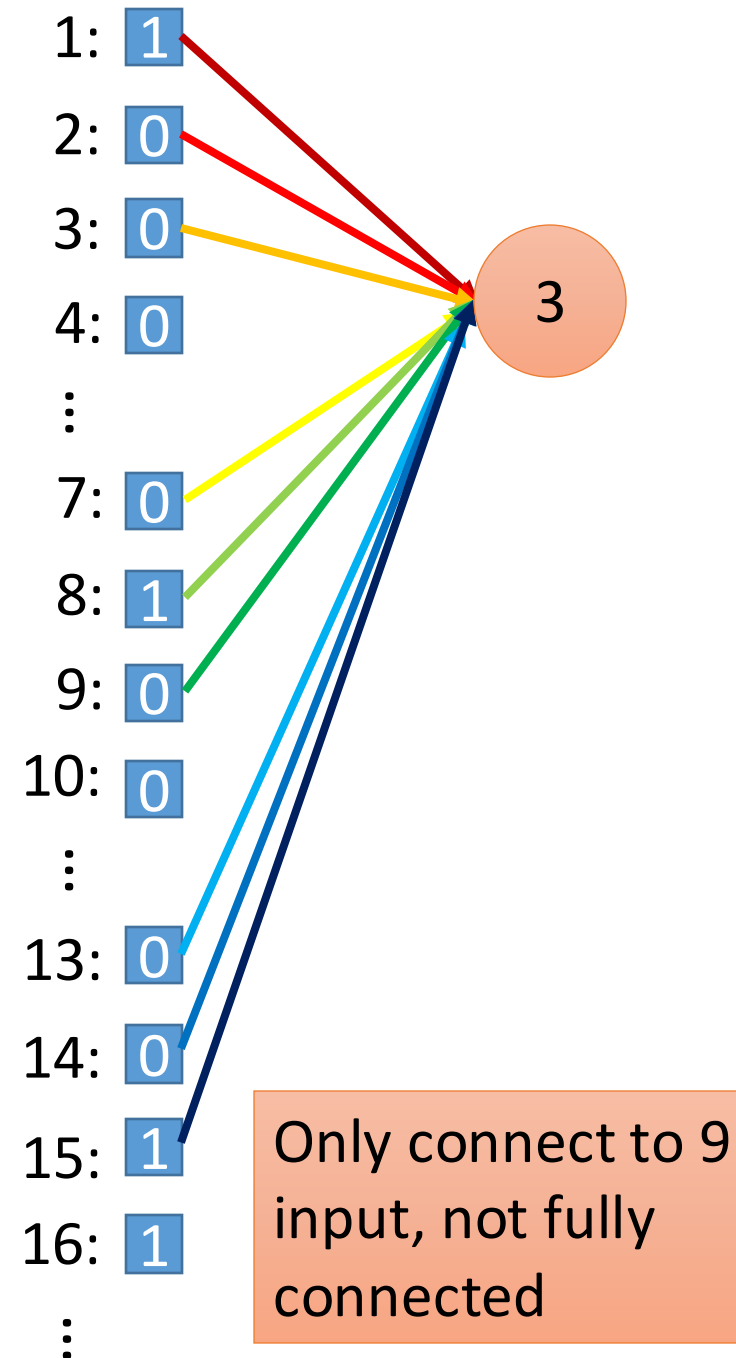
Fully-
connected

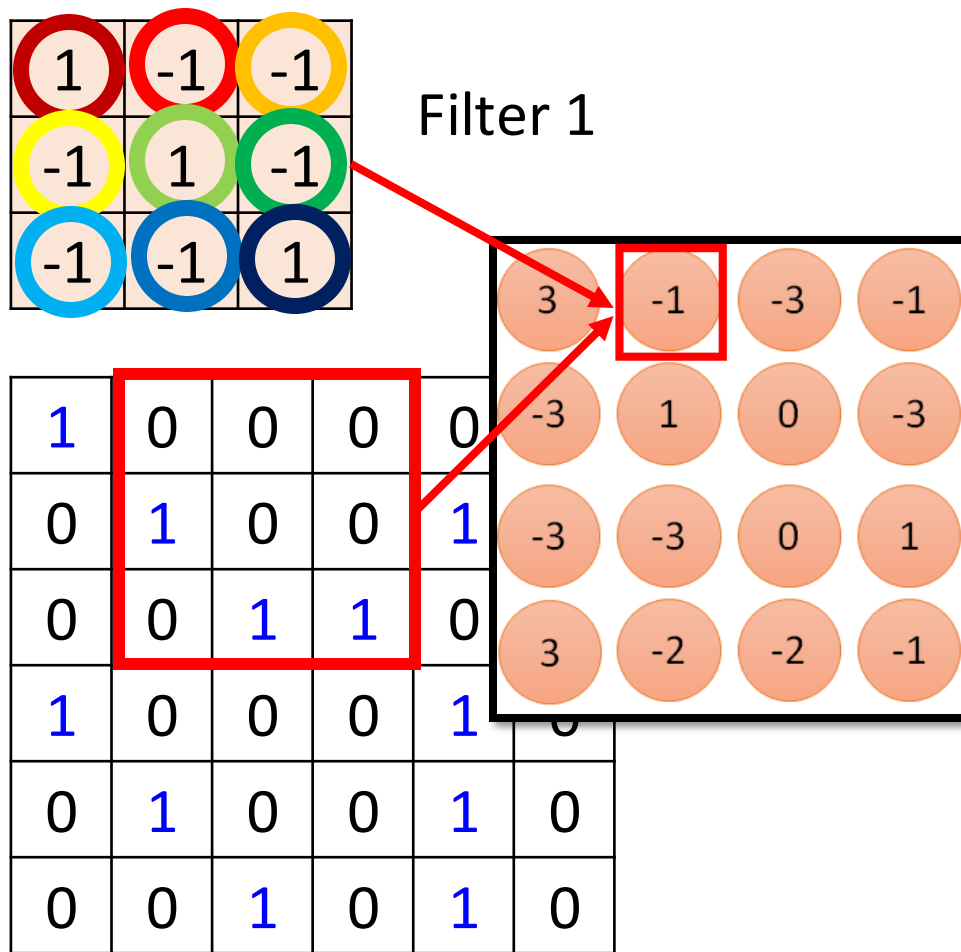
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0





Less parameters!

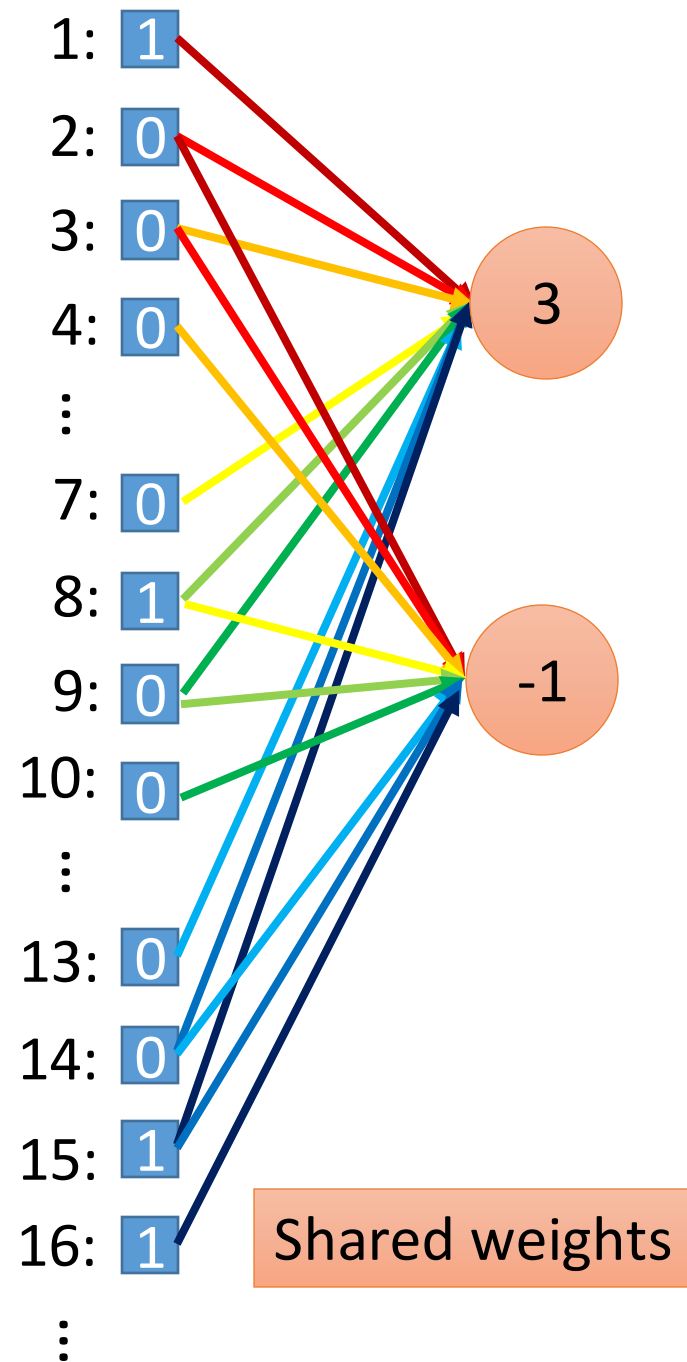




6 x 6 image

Less parameters!

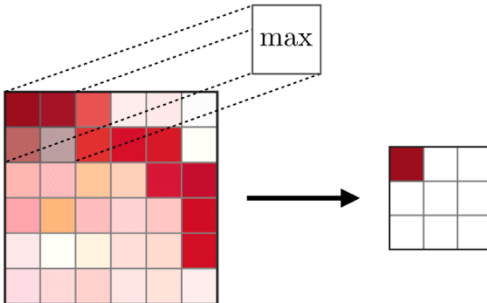
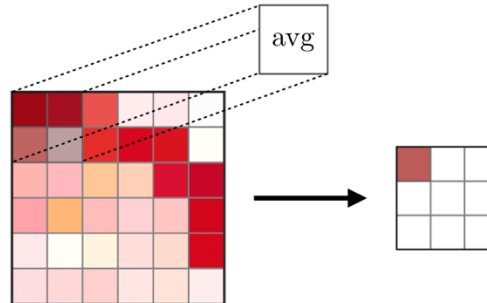
Even less parameters!



Agenda

- Convolutional Neural Networks (CNN)
 - What is CNN
 - Convolutional Layer
 - Pooling Layer
 - Backpropagation

Pooling layer

Type	Max pooling	Average pooling
Purpose	Each pooling operation selects the maximum value of the current view	Each pooling operation averages the values of the current view
Illustration		
Comments	• Preserves detected features • Most commonly used	• Downsamples feature map • Used in LeNet

The pooling layer is a downsampling operation, typically applied after a convolution layer !

CNN – max pooling

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

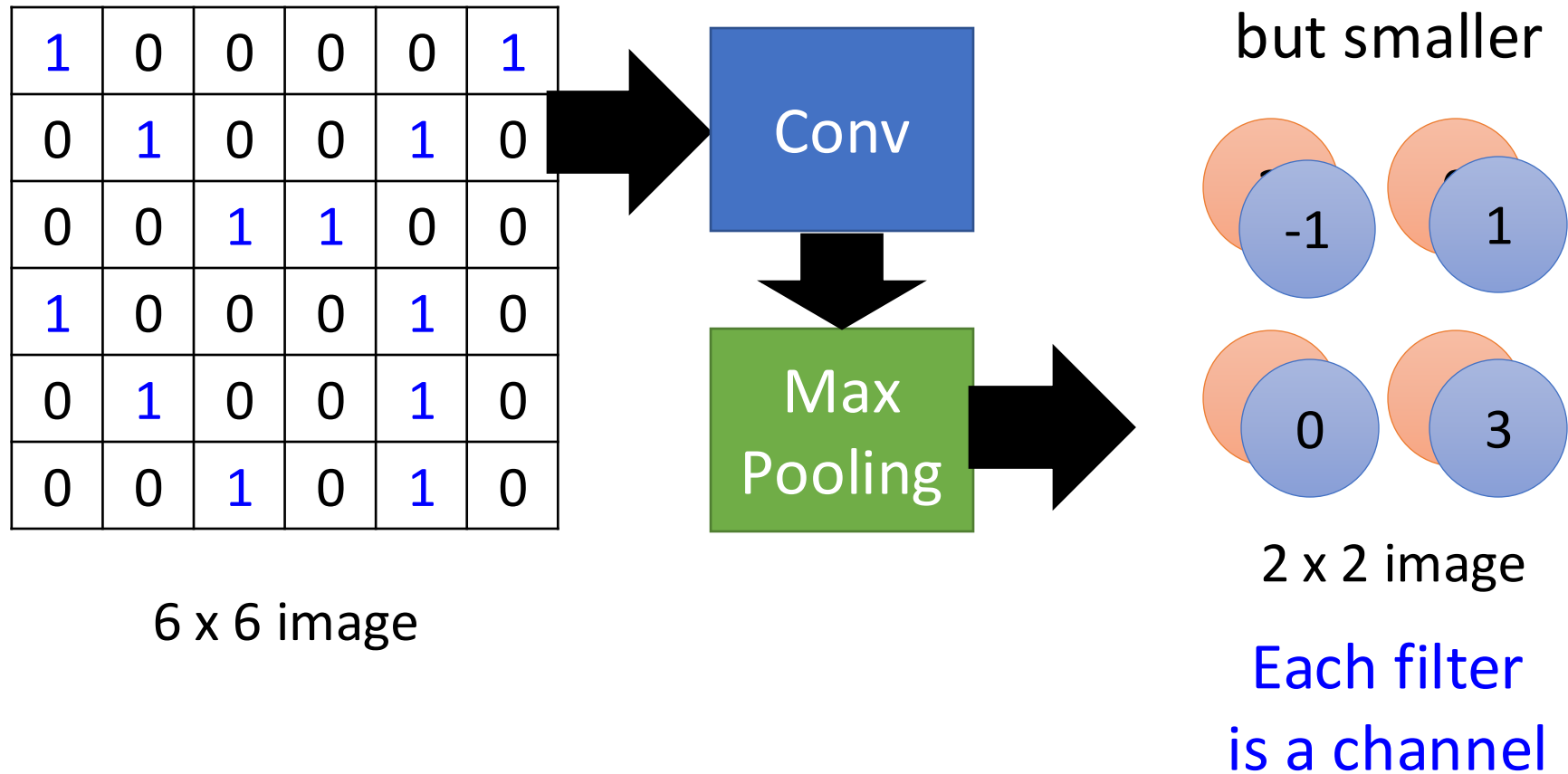
-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

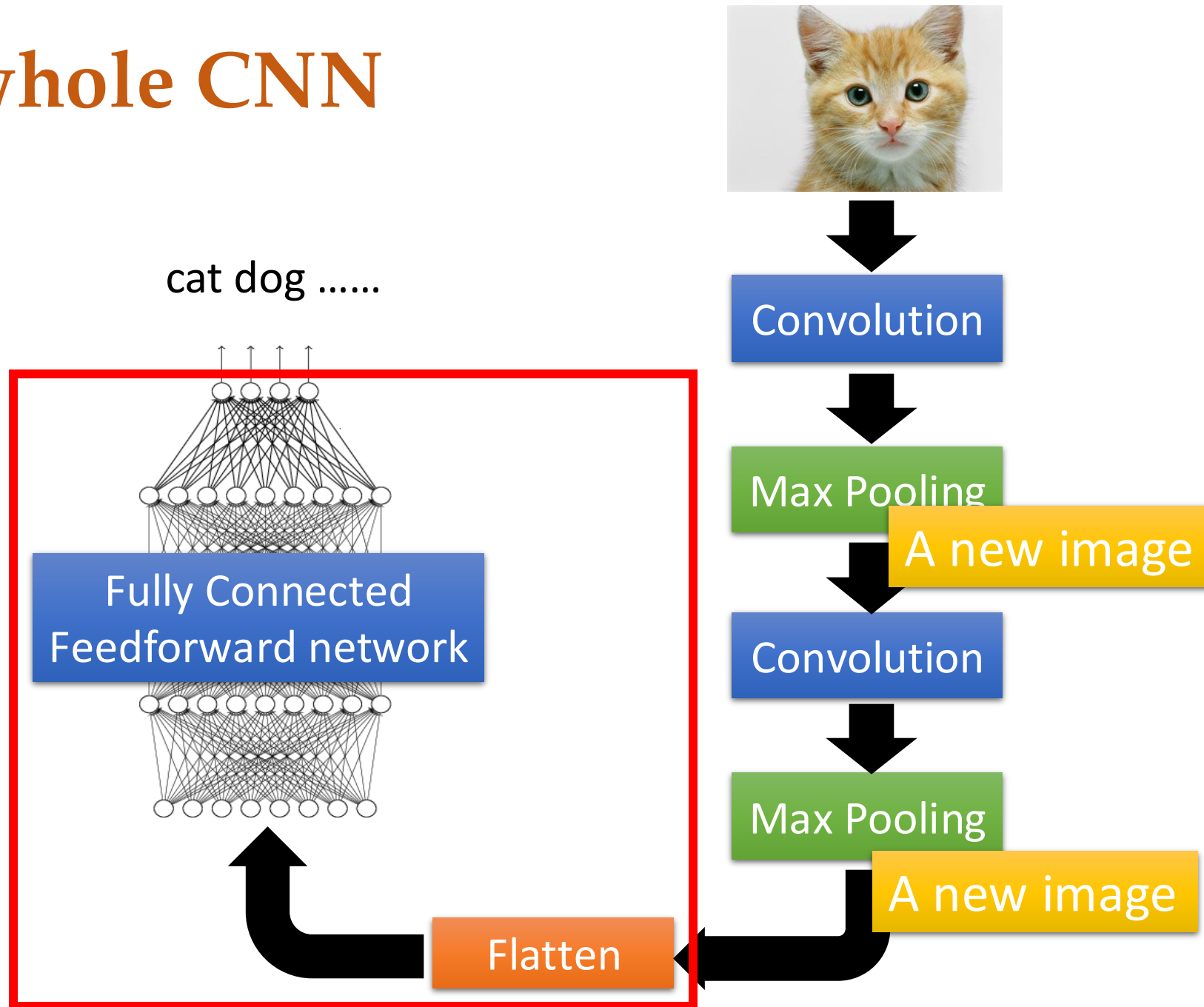
3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

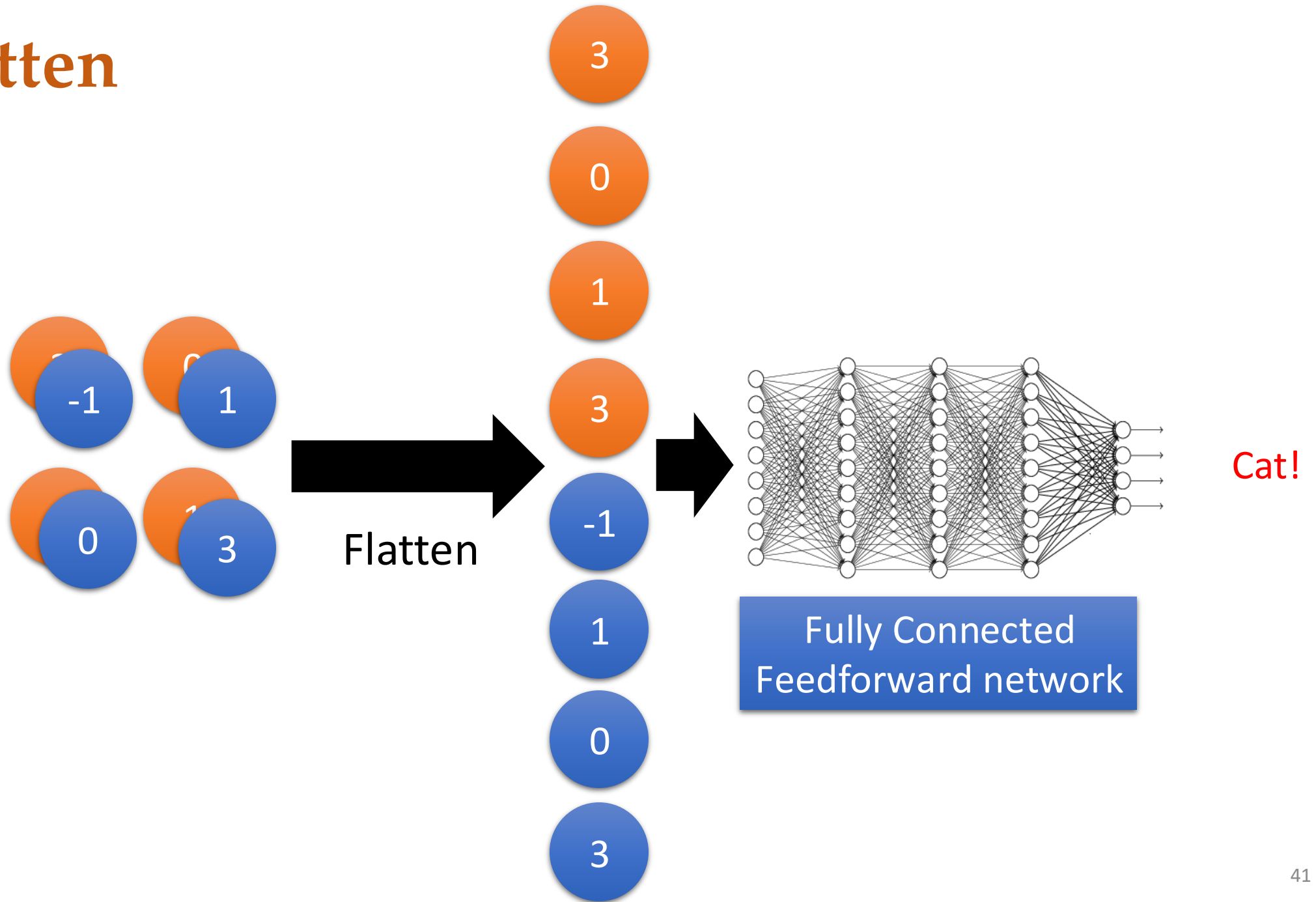
CNN – max pooling



The whole CNN



Flatten

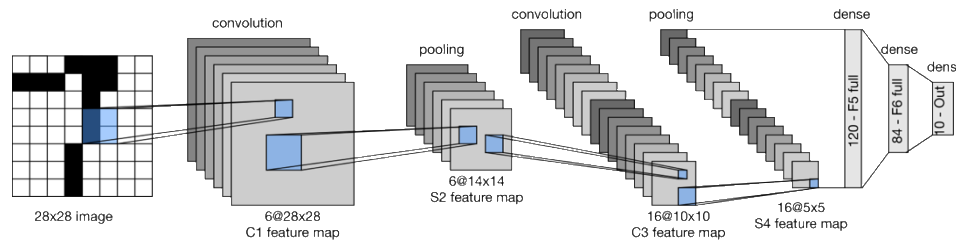


Agenda

- Convolutional Neural Networks (CNN)
 - What is CNN
 - Convolutional Layer
 - Pooling Layer
 - Backpropagation

Example: LeNet

LeCun, Yann, et al. "Gradient-based learning applied to document recognition."



Layer	Output Size	Weight Size
Input	1 x 28 x 28	
Conv (Cout=6, K=5, P=2, S=1)	6 x 28 x 28	6 x 1 x 5 x 5
ReLU	6 x 28 x 28	
MaxPool(K=2, S=2)	6 x 14 x 14	
Conv (Cout=16, K=5, S=1)	16 x 10 x 10	16 x 6 x 5 x 5
ReLU	16 x 10 x 10	
MaxPool(K=2, S=2)	16 x 5 x 5	
Flatten	16*5*5	
Linear (400 -> 120)	120	400 x 120
ReLU	120	
Linear (120 -> 84)	84	120 x 84
ReLU	120	
Linear (84 -> 10)	10	84 x 10

net = nn.Sequential(

```
nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5, stride=1,
padding=2),
```

```
nn.relu(),
```

```
nn.MaxPool2d(kernel_size=2, stride=2),
```

```
nn.Conv2d(in_channels=6, out_channels= 16, kernel_size=5, stride=1,
padding=0),
```

```
nn.relu(),
```

```
nn.MaxPool2d(kernel_size=2, stride=2),
```

```
nn.Flatten(),
```

```
nn.Linear(16*5*5, 120),
```

```
nn.relu(),
```

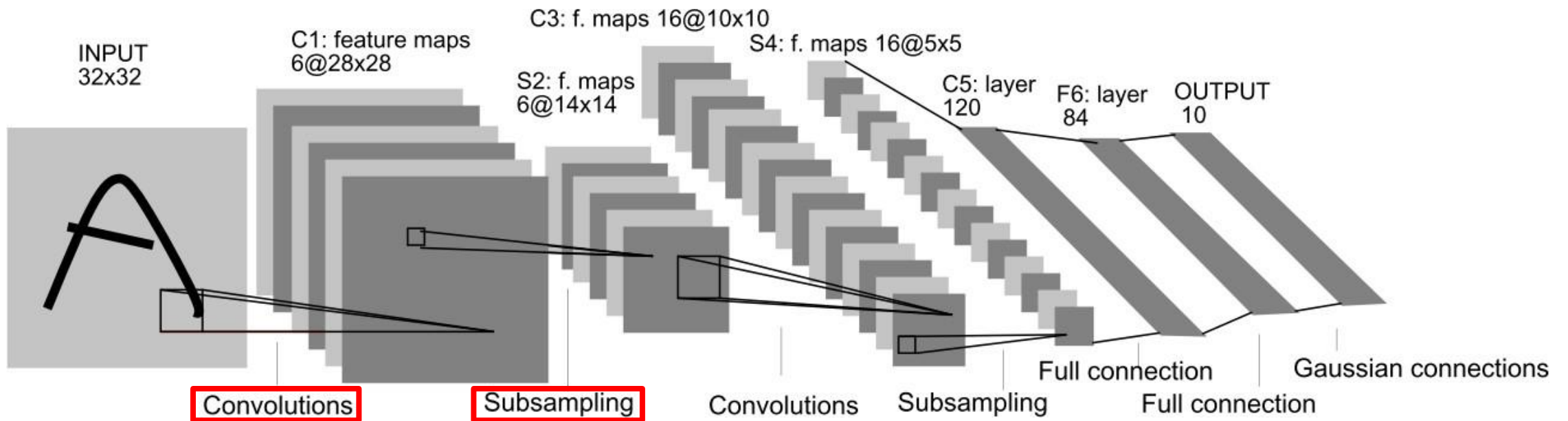
```
nn.Linear(120, 84),
```

```
nn.relu(),
```

```
nn.Linear(84, 10))
```

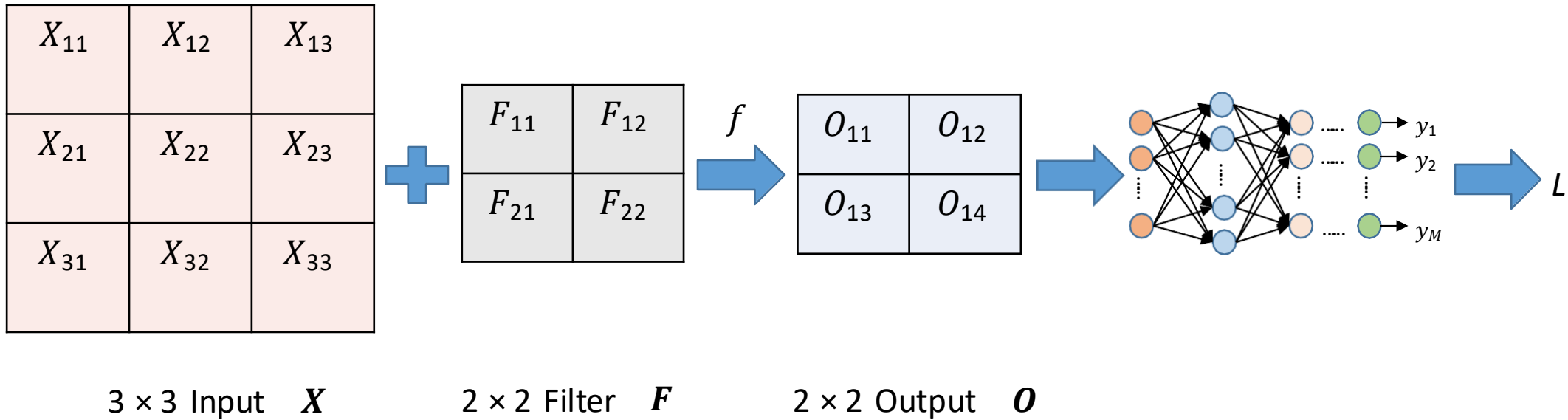
Backpropagation

A simple but important CNN – LeNet 5



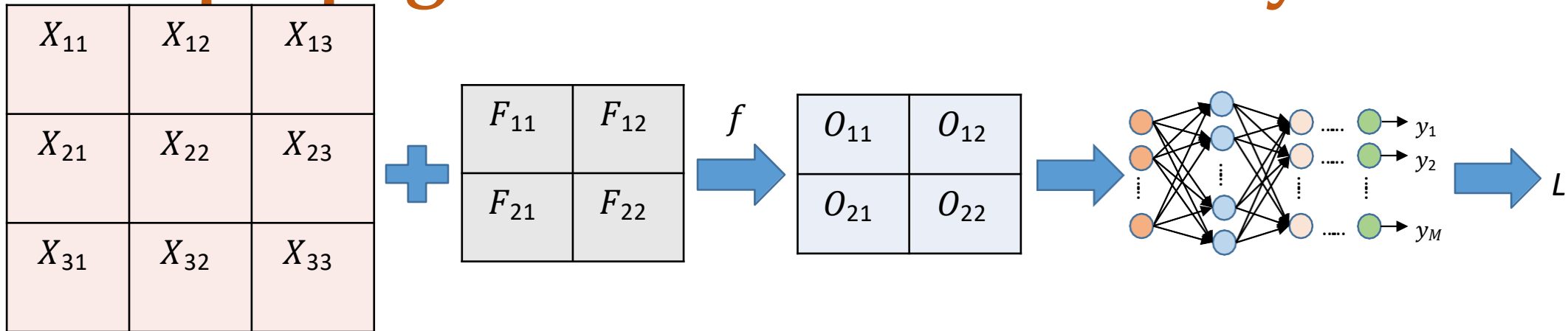
- Back-propagation – Chain rule
- How to compute the gradients of convolution layers and pooling layers?

Backpropagation – Convolution Layer



- The output of convolution operation $O = f(F, X)$
- The loss L
- Assume that we have already computed $\frac{\partial L}{\partial O_{ij}}$

Backpropagation – Convolution Layer



- How to compute $\frac{\partial L}{\partial F_{11}}$

Chain Rule



$$\frac{\partial L}{\partial F_{11}} = \left(\frac{\partial L}{\partial \mathbf{O}} \right)^T \frac{\partial \mathbf{O}}{\partial F_{11}}$$

$$\frac{\partial L}{\partial \mathbf{O}} = \nabla L(O_{11}, O_{12}, O_{21}, O_{22})$$

$$= \left(\frac{\partial L}{\partial O_{11}}, \frac{\partial L}{\partial O_{12}}, \frac{\partial L}{\partial O_{21}}, \frac{\partial L}{\partial O_{22}} \right)^T$$

$$\mathbf{O} = f(\mathbf{F}, \mathbf{X}) = (O_{11}, O_{12}, O_{21}, O_{22})^T \triangleq (f_1(\mathbf{F}, \mathbf{X}), f_2(\mathbf{F}, \mathbf{X}), f_3(\mathbf{F}, \mathbf{X}), f_4(\mathbf{F}, \mathbf{X}))^T$$

$$\frac{\partial \mathbf{O}}{\partial F_{11}} = \left(\frac{\partial f_1}{\partial F_{11}}, \frac{\partial f_2}{\partial F_{11}}, \frac{\partial f_3}{\partial F_{11}}, \frac{\partial f_4}{\partial F_{11}} \right)^T$$

Backpropagation – Convolution Layer

X_{11}	X_{12}	X_{13}
X_{21}	X_{22}	X_{23}
X_{31}	X_{32}	X_{33}



F_{11}	F_{12}
F_{21}	F_{22}



O_{11}	O_{12}
O_{21}	O_{22}

$$\frac{\partial L}{\partial F_{11}} = \frac{\partial L}{\partial O_{11}} \frac{\partial f_1}{\partial F_{11}} + \frac{\partial L}{\partial O_{12}} \frac{\partial f_2}{\partial F_{11}} + \frac{\partial L}{\partial O_{21}} \frac{\partial f_3}{\partial F_{11}} + \frac{\partial L}{\partial O_{22}} \frac{\partial f_4}{\partial F_{11}}$$

$$\frac{\partial L}{\partial F_{12}} = \frac{\partial L}{\partial O_{11}} \frac{\partial f_1}{\partial F_{12}} + \frac{\partial L}{\partial O_{12}} \frac{\partial f_2}{\partial F_{12}} + \frac{\partial L}{\partial O_{21}} \frac{\partial f_3}{\partial F_{12}} + \frac{\partial L}{\partial O_{22}} \frac{\partial f_4}{\partial F_{12}}$$

$$\frac{\partial L}{\partial F_{21}} = \frac{\partial L}{\partial O_{11}} \frac{\partial f_1}{\partial F_{21}} + \frac{\partial L}{\partial O_{12}} \frac{\partial f_2}{\partial F_{21}} + \frac{\partial L}{\partial O_{21}} \frac{\partial f_3}{\partial F_{21}} + \frac{\partial L}{\partial O_{22}} \frac{\partial f_4}{\partial F_{21}}$$

$$\frac{\partial L}{\partial F_{22}} = \frac{\partial L}{\partial O_{11}} \frac{\partial f_1}{\partial F_{22}} + \frac{\partial L}{\partial O_{12}} \frac{\partial f_2}{\partial F_{22}} + \frac{\partial L}{\partial O_{21}} \frac{\partial f_3}{\partial F_{22}} + \frac{\partial L}{\partial O_{22}} \frac{\partial f_4}{\partial F_{22}}$$

Backpropagation – Convolution Layer

$$\mathbf{O} = f(\mathbf{F}; \mathbf{X}) \quad f_1(\mathbf{X}) = O_{11} = F_{11}X_{11} + F_{12}X_{12} + F_{21}X_{21} + F_{22}X_{22}$$

$$f_2(\mathbf{X}) = O_{12} = F_{11}X_{12} + F_{12}X_{13} + F_{21}X_{22} + F_{22}X_{23}$$

$$f_3(\mathbf{X}) = O_{21} = F_{11}X_{21} + F_{12}X_{22} + F_{21}X_{31} + F_{22}X_{32}$$

$$f_4(\mathbf{X}) = O_{22} = F_{11}X_{22} + F_{12}X_{23} + F_{21}X_{32} + F_{22}X_{33}$$

X_{11}	X_{12}	X_{13}
X_{21}	X_{22}	X_{23}
X_{31}	X_{32}	X_{33}



F_{11}	F_{12}
F_{21}	F_{22}

$$\frac{\partial f_1}{\partial F_{11}} = X_{11}$$

⋮

$$\frac{\partial f_1}{\partial F_{22}} = X_{22}$$

$$\frac{\partial f_2}{\partial F_{11}} = X_{12}$$

⋮

$$\frac{\partial f_2}{\partial F_{22}} = X_{23}$$

$$\frac{\partial f_3}{\partial F_{11}} = X_{21}$$

⋮

$$\frac{\partial f_3}{\partial F_{22}} = X_{32}$$

$$\frac{\partial f_4}{\partial F_{11}} = X_{22}$$

⋮

$$\frac{\partial f_4}{\partial F_{22}} = X_{33}$$

Backpropagation – Convolution Layer

X_{11}	X_{12}	X_{13}
X_{21}	X_{22}	X_{23}
X_{31}	X_{32}	X_{33}



$\frac{\partial L}{\partial O_{11}}$	$\frac{\partial L}{\partial O_{12}}$
$\frac{\partial L}{\partial O_{21}}$	$\frac{\partial L}{\partial O_{22}}$



$\frac{\partial L}{\partial F_{11}}$	$\frac{\partial L}{\partial F_{12}}$
$\frac{\partial L}{\partial F_{21}}$	$\frac{\partial L}{\partial F_{22}}$

$$\frac{\partial L}{\partial F_{11}} = \frac{\partial L}{\partial O_{11}} X_{11} + \frac{\partial L}{\partial O_{12}} X_{12} + \frac{\partial L}{\partial O_{21}} X_{21} + \frac{\partial L}{\partial O_{22}} X_{22}$$

$$\frac{\partial L}{\partial F_{12}} = \frac{\partial L}{\partial O_{11}} X_{12} + \frac{\partial L}{\partial O_{12}} X_{13} + \frac{\partial L}{\partial O_{21}} X_{22} + \frac{\partial L}{\partial O_{22}} X_{23}$$

$$\frac{\partial L}{\partial F_{21}} = \frac{\partial L}{\partial O_{11}} X_{21} + \frac{\partial L}{\partial O_{12}} X_{22} + \frac{\partial L}{\partial O_{21}} X_{31} + \frac{\partial L}{\partial O_{22}} X_{32}$$

$$\frac{\partial L}{\partial F_{22}} = \frac{\partial L}{\partial O_{11}} X_{22} + \frac{\partial L}{\partial O_{12}} X_{23} + \frac{\partial L}{\partial O_{21}} X_{32} + \frac{\partial L}{\partial O_{22}} X_{33}$$

Backpropagation – Convolution Layer

- How to compute $\frac{\partial L}{\partial X_{11}}$?

Chain Rule



$$\frac{\partial L}{\partial X_{11}} = \left(\frac{\partial L}{\partial \mathbf{O}} \right)^T \frac{\partial \mathbf{O}}{\partial X_{11}}$$

$$\frac{\partial L}{\partial \mathbf{O}} = \nabla L(O_{11}, O_{12}, O_{21}, O_{22})$$

$$= \left(\frac{\partial L}{\partial O_{11}}, \frac{\partial L}{\partial O_{12}}, \frac{\partial L}{\partial O_{21}}, \frac{\partial L}{\partial O_{22}} \right)^T$$

$$\mathbf{O} = f(\mathbf{F}, \mathbf{X}) = (O_{11}, O_{12}, O_{21}, O_{22})^T \triangleq (f_1(\mathbf{F}, \mathbf{X}), f_2(\mathbf{F}, \mathbf{X}), f_3(\mathbf{F}, \mathbf{X}), f_4(\mathbf{F}, \mathbf{X}))^T$$

$$\frac{\partial \mathbf{O}}{\partial X_{11}} = \left(\frac{\partial f_1}{\partial X_{11}}, \frac{\partial f_2}{\partial X_{11}}, \frac{\partial f_3}{\partial X_{11}}, \frac{\partial f_4}{\partial X_{11}} \right)^T$$

Backpropagation – Convolution Layer

$$\mathbf{O} = f(\mathbf{F}; \mathbf{X}) \quad f_1(\mathbf{X}) = O_{11} = F_{11}X_{11} + F_{12}X_{12} + F_{21}X_{21} + F_{22}X_{22}$$

$$f_2(\mathbf{X}) = O_{12} = F_{11}X_{12} + F_{12}X_{13} + F_{21}X_{22} + F_{22}X_{23}$$

$$f_3(\mathbf{X}) = O_{21} = F_{11}X_{21} + F_{12}X_{22} + F_{21}X_{31} + F_{22}X_{32}$$

$$f_4(\mathbf{X}) = O_{22} = F_{11}X_{22} + F_{12}X_{23} + F_{21}X_{32} + F_{22}X_{33}$$

$$\frac{\partial L}{\partial X_{11}} = \frac{\partial L}{\partial O_{11}} F_{11} + \frac{\partial L}{\partial O_{12}} 0 + \frac{\partial L}{\partial O_{21}} 0 + \frac{\partial L}{\partial O_{22}} 0$$

$$\frac{\partial L}{\partial X_{12}} = \frac{\partial L}{\partial O_{11}} F_{12} + \frac{\partial L}{\partial O_{12}} F_{11} + \frac{\partial L}{\partial O_{21}} 0 + \frac{\partial L}{\partial O_{22}} 0$$

$$\frac{\partial L}{\partial X_{13}} = \frac{\partial L}{\partial O_{11}} 0 + \frac{\partial L}{\partial O_{12}} F_{12} + \frac{\partial L}{\partial O_{21}} 0 + \frac{\partial L}{\partial O_{22}} 0$$

$$\frac{\partial L}{\partial X_{21}} = \frac{\partial L}{\partial O_{11}} F_{21} + \frac{\partial L}{\partial O_{12}} 0 + \frac{\partial L}{\partial O_{21}} F_{11} + \frac{\partial L}{\partial O_{22}} 0$$

$$\frac{\partial L}{\partial X_{22}} = \frac{\partial L}{\partial O_{11}} F_{22} + \frac{\partial L}{\partial O_{12}} F_{21} + \frac{\partial L}{\partial O_{21}} F_{12} + \frac{\partial L}{\partial O_{22}} F_{11}$$

$$\frac{\partial L}{\partial X_{23}} = \frac{\partial L}{\partial O_{11}} 0 + \frac{\partial L}{\partial O_{12}} F_{22} + \frac{\partial L}{\partial O_{21}} 0 + \frac{\partial L}{\partial O_{22}} F_{11}$$

$$\frac{\partial L}{\partial X_{31}} = \frac{\partial L}{\partial O_{11}} 0 + \frac{\partial L}{\partial O_{12}} 0 + \frac{\partial L}{\partial O_{21}} F_{21} + \frac{\partial L}{\partial O_{22}} 0$$

$$\frac{\partial L}{\partial X_{32}} = \frac{\partial L}{\partial O_{11}} 0 + \frac{\partial L}{\partial O_{12}} 0 + \frac{\partial L}{\partial O_{21}} F_{22} + \frac{\partial L}{\partial O_{22}} F_{21}$$

$$\frac{\partial L}{\partial X_{33}} = \frac{\partial L}{\partial O_{11}} 0 + \frac{\partial L}{\partial O_{12}} 0 + \frac{\partial L}{\partial O_{21}} 0 + \frac{\partial L}{\partial O_{22}} F_{22}$$

Backpropagation – Pooling Layer

- Max Pooling

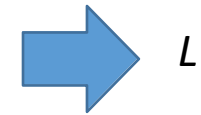
X_{11}^i	X_{12}^i	X_{13}^i	X_{14}^i
X_{21}^i	X_{22}^i	X_{23}^i	X_{24}^i
X_{31}^i	X_{32}^i	X_{33}^i	X_{34}^i
X_{41}^i	X_{42}^i	X_{43}^i	X_{44}^i

4 × 4 Input



X_{11}^o	X_{23}^o
X_{42}^o	X_{33}^o

2 × 2 output



L

Only need to compute the gradients of the inputs

$$\frac{\partial L}{\partial X_{11}^i} = \frac{\partial L}{\partial X_{11}^o}$$

$$\frac{\partial L}{\partial X_{12}^i} = 0$$

$$\frac{\partial L}{\partial X_{21}^i} = 0$$

$$\frac{\partial L}{\partial X_{22}^i} = 0$$

$$\frac{\partial L}{\partial X_{13}^i} = 0$$

$$\frac{\partial L}{\partial X_{14}^i} = 0$$

$$\frac{\partial L}{\partial X_{23}^i} = \frac{\partial L}{\partial X_{23}^o}$$

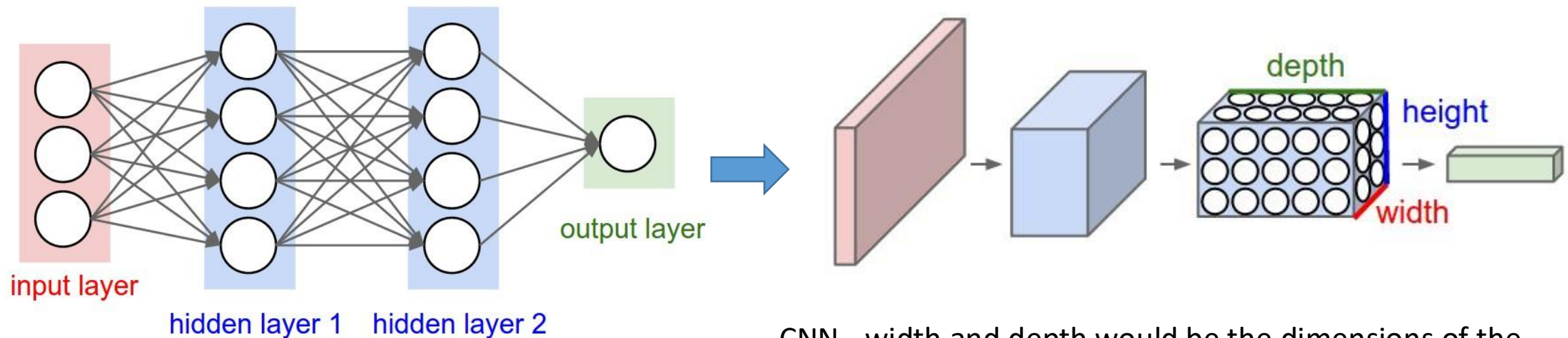
$$\frac{\partial L}{\partial X_{24}^i} = 0$$

⋮

We assign the error to where it comes from

Why CNN for image learning

- Comparing to record data where each data point is a vector
 - Image data is a matrix with **grid-like topology**
 - Image data is a tensor consisting of **multiple channels (RGB)**
 - Image data is a high-dimensional data require **computational efficiency**



Fully-connected layers

CNN - width and depth would be the dimensions of the image, and height would be 3 (Red, Green, Blue channels)

[Fei-Fei Li, Stanford CS 231](#)

Conv layer in PyTorch

```
torch.nn.Conv2d(in_channels: int, out_channels: int, kernel_size: Union[T, Tuple[T, T]], stride: Union[T, Tuple[T, T]] = 1, padding: Union[T, Tuple[T, T]] = 0, dilation: Union[T, Tuple[T, T]] = 1, groups: int = 1, bias: bool = True, padding_mode: str = 'zeros')
```

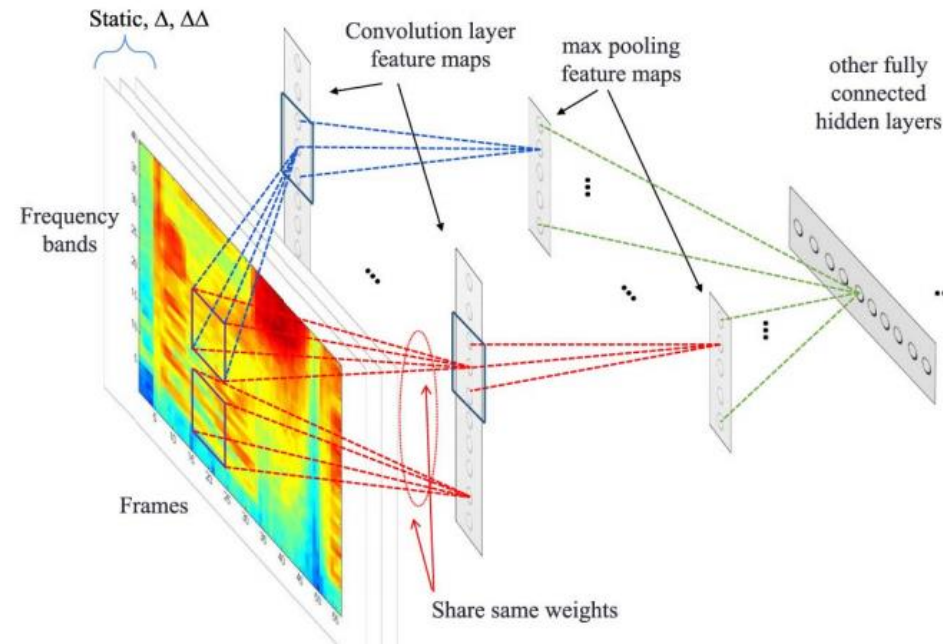
```
nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5, stride=1, padding=2)
```

- `in_channels` ([int](#)) – Number of channels in the input image
- `out_channels` ([int](#)) – Number of channels produced by the convolution
- `kernel_size` ([int](#) or [tuple](#)) – Size of the convolving kernel
- `stride` ([int](#) or [tuple](#), *optional*) – Stride of the convolution. Default: 1
- `padding` ([int](#) or [tuple](#), *optional*) – Zero-padding added to both sides of the input. Default: 0
- `padding_mode` (*string, optional*) – 'zeros', 'reflect', 'replicate' or 'circular'. Default: 'zeros'
- `dilation` ([int](#) or [tuple](#), *optional*) – Spacing between kernel elements. Default: 1
- `groups` ([int](#), *optional*) – Number of blocked connections from input channels to output channels. Default: 1
- `bias` ([bool](#), *optional*) – If True, adds a learnable bias to the output. Default: True

More applications of CNN

Speech

<https://dl.acm.org/doi/10.1109/TASLP.2014.2339736>



Natural Language Processing

<https://www.aclweb.org/anthology/S15-2079/>

